HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1

TIỂU LUẬN FINAL

PHÁT TRIỂN CÁC HỆ THỐNG THÔNG MINH

TỪ HỌC MÁY ĐẾN HỆ ĐA TÁC TỬ

Lý thuyết thực nghiệm và ứng dụng DiaAgents

Giảng viên hướng dẫn	PGS.TS Trần Đình Quế
Sinh viên thực hiện	Nguyễn Minh Vinh
Mã sinh viên	B23DCCN934
Lớp	D23CTPM01
Nhóm	08

Hà Nội, tháng 11 năm 2026

MỤC LỤC

TÓM TẮT 5
MỞ ĐẦU 6
0.1 Bối cảnh và mục tiêu 6
0.2 Phạm vi dữ liệu và sản phẩm 6
0.3 Nguyên tắc thực nghiệm 7
CHƯƠNG 1. LỊCH SỬ PHÁT TRIỂN AI 8
1.1. Nền tảng 8
1.2. Turing và Dartmouth 8
1.3. AI biểu tượng, hệ chuyên gia và giới hạn 9
1.4. Từ đặc trưng thủ công đến học biểu diễn 10
1.5. CNN, RNN và attention trong dòng phát triển 11
1.6. Dataset minh họa: CDC Diabetes 11
1.7. Dataset minh họa: MNIST 12
1.8. Dataset minh họa: AAPL 13
CHƯƠNG 2. ML CƠ BẢN 15
2.1. Học có giám sát 15
2.2. Các phương pháp tuyến tính và dựa trên khoảng cách 15
2.3. Cây, ensemble và mạng MLP 16
2.4. Dữ liệu và tiền xử lý: CDC Diabetes 17
2.5. Dữ liệu và tiền xử lý: Giá nhà Việt Nam 17
2.6. Cài đặt MLP scratch và đạo hàm 18
2.7. MLP Keras và PyTorch tương ứng 19
2.8. Kết quả và baseline: CDC Diabetes 19
2.9. Kết quả và baseline: Giá nhà Việt Nam 20
2.10. Learning curve của hai bài ML 21
2.11. Phân bố, ví dụ sai và chi phí 22
2.12. Tiểu kết và giới hạn ML 23
CHƯƠNG 3. CNN 24
3.1. Động cơ và cấu trúc ảnh 24
3.2. Convolution, pooling và số tham số 24

3.3. Loss và lan truyền ngược qua CNN 25
3.4. Dataset ảnh: MNIST 25
3.5. Dataset ảnh: EuroSAT 26
3.6. CNN scratch: forward và cập nhật 27
3.7. CNN Keras và PyTorch 28
3.8. Đánh giá CNN trên MNIST 28
3.9. Đánh giá CNN trên EuroSAT 29
3.10. Đường học và chi phí CNN 30
3.11. Phân tích trường hợp sai 31
3.12. So sánh các kiến trúc khác nhau 32
3.13. Kết quả so sánh kiến trúc trên hai dataset 32
3.14. Tiểu kết CNN và hướng mở rộng 34
CHƯƠNG 4. RNN 35
4.1. Chuỗi và trạng thái 35
4.2. Vanilla RNN và hợp hàm theo thời gian 35
4.3. BPTT và kiểm soát gradient 36
4.4. LSTM, GRU và phạm vi mô hình 36
4.5. Dataset chuỗi: AAPL 37
4.6. Dataset chuỗi: Online Retail II 38
4.7. RNN Keras và PyTorch 38
4.8. Kết quả RNN: AAPL 39
4.9. Kết quả RNN: Online Retail II 40
4.10. Learning curve và chi phí RNN 41
4.11. Phân tích sai số và dịch chuyển thời gian 41
4.12. So sánh các kiến trúc khác nhau 42
4.13. Kết quả so sánh kiến trúc trên hai dataset 43
4.14. Tiểu kết RNN 44
CHƯƠNG 5. KNOWLEDGE BASE, KNOWLEDGE GRAPH VÀ RETRIEVAL-AUGMENTED GENERATION 46
5.1. Mở đầu 46
5.2. Knowledge Base và Knowledge Graph 46
5.3. Retrieval-Augmented Generation 47

5.4. Case study KB/RAG cho bệnh đái tháo đường 48
5.5. Case study KB/RAG cho AAPL và báo cáo doanh nghiệp 49
5.6. Hợp các hàm và contract dùng chung 49
5.7. Cầu nối sang hệ đa tác tử 50
5.8. Đánh giá và tiểu kết 51
5.9. Mẫu dữ liệu và đường dẫn hồ sơ 53
5.10. Triển khai ngắn 53
CHƯƠNG 6 HỌC SÂU TRÊN ĐỒ THỊ 54
6.1 Dữ liệu đồ thị và bài toán học 54
6.2 Từ mạng nhiều lớp đến truyền thông điệp 54
6.3 Dữ liệu CDC và cách tạo đồ thị 55
6.4 Cài đặt và lựa chọn mô hình 56
6.5 Kết quả và nhận xét 56
6.6 Notebook và tái lập 57
6.7 Tiểu kết và giới hạn 57
CHƯƠNG 7 HỆ ĐA TÁC TỬ 58
7.1 Từ tác tử đơn đến hệ đa tác tử 58
7.2 Kế thừa Agent trong tiểu luận 3 58
7.3 Minh họa learning agent từ bài Agent 01 58
7.4 Dữ liệu và nhiệm vụ của DiaAgents 59
7.5 Mô hình ML và deep learning 59
7.6 Neo4j Knowledge Base và GraphRAG 60
7.7 Điều phối và phản biện 60
7.8 Học tăng cường cho bước quy trình 61
7.9 So sánh Single Agent và Multi-Agent 61
7.10 Notebook và cách demo 62
7.11 Triển khai và phạm vi hoàn thành 64
7.12 Tiểu kết 65
KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 66

TÓM TẮT

Tiểu luận tổng hợp quá trình xây dựng hệ thống thông minh từ biểu diễn dữ liệu, học mô hình đến truy xuất tri thức và điều phối tác tử. Quyển Final kế thừa các thực nghiệm của tiểu luận 3: đối chiếu cài đặt scratch, Keras và PyTorch cho dữ liệu bảng, ảnh và chuỗi thời gian; sau đó phát triển thêm học sâu trên đồ thị và ứng dụng đa tác tử. Mục tiêu là nối công thức, mã nguồn, kết quả thực nghiệm và hành vi demo trong một quy trình có thể kiểm tra.

Các chương đầu giữ dữ liệu CDC BRFSS 2015, giá nhà Việt Nam, MNIST, EuroSAT, AAPL và Online Retail II của sinh viên. Chương 6 xây đồ thị tương đồng từ một tập con CDC, so sánh MLP, GCN và GraphSAGE. Chương 7 phát triển hệ Agent sử dụng Diabetes Prediction Dataset và CDC BRFSS của project hiện tại, kết hợp mô hình PyTorch, mô hình sklearn đối chiếu, Neo4j, truy xuất bằng chứng, trao đổi thông điệp và Q-learning cho quy trình.

Thực nghiệm phân biệt hiệu quả mô hình với hiệu quả kiến trúc phần mềm. Một F1 cao chỉ phản ánh bộ dữ liệu và cách chia đã xác định; một trace nhiều bước chỉ thể hiện khả năng quan sát, không tự chứng minh câu trả lời tốt hơn. Hai hệ Single và Multi được so trên cùng 58 tình huống và cùng công cụ. Các mô hình, số đo và dataset của bài tham khảo không được dùng thay cho kết quả của sinh viên.

MỞ ĐẦU

0.1 Bối cảnh và mục tiêu

Một ứng dụng AI hoàn chỉnh cần nhiều hơn một mô hình dự đoán. Dữ liệu phải được kiểm tra, đặc trưng và nhãn phải được định nghĩa, tri thức phải có nguồn và đầu ra phải được đặt trong phạm vi phù hợp. Tiểu luận tiếp cận vấn đề theo từng lớp: ML xử lý dữ liệu bảng; CNN khai thác cấu trúc ảnh; RNN mô hình hóa chuỗi; KG và RAG biểu diễn bằng chứng; GNN học trên cấu trúc liên kết; Agent điều phối các công cụ đó.

Mục tiêu của quyển Final là tiếp nối kết quả đã có thay vì thay toàn bộ bài bằng một miền mới. Mỗi chương giữ đường dẫn thực nghiệm, nguồn dữ liệu và các điều kiện đánh giá. Nội dung mới được viết thành chương hoàn chỉnh, đồng thời phần kết luận được cập nhật theo các kết quả hiện có.

0.2 Phạm vi dữ liệu và sản phẩm

Chương 1–5 kế thừa tiểu luận 3; phần Agent trước đây ở Chương 6 được đặt lại trong mạch phát triển của Chương 7. Chương 6 mới trình bày học sâu trên đồ thị để tương ứng cấu trúc bảy chương của bài tham khảo. Dữ liệu đồ thị ở đây được xây từ CDC của sinh viên, không dùng Pubmed-Diabetes hoặc các bộ NHANES, RetinaMNIST, ShanghaiT1DM của bài mẫu.

Mã nguồn và notebook Chương 1–5: https://github.com/Vinhdiesel28/tieuluan3 . Mã nguồn và notebook phần phát triển mới: https://github.com/Vinhdiesel28/final . Demo Agent cục bộ: http://127.0.0.1:8010 . Địa chỉ localhost chỉ hoạt động trên máy đang chạy server, không phải liên kết triển khai công khai.

Chương	Dữ liệu của sinh viên	Sản phẩm kiểm chứng
1	Các dataset đại diện của bài	Notebook lịch sử và thống kê dữ liệu
2	CDC Diabetes và giá nhà Việt Nam	MLP scratch/Keras/PyTorch và baseline ML
3	MNIST và EuroSAT	CNN và so sánh kiến trúc
4	AAPL và Online Retail II	RNN và so sánh kiến trúc
5	Corpus CDC và AAPL của tiểu luận 3	12 tài liệu, 23 quan hệ, benchmark 15 câu
6	1.600 mẫu từ CDC BRFSS 2015	MLP, GCN, GraphSAGE và notebook 10
7	Diabetes Prediction, CDC và KB project	Notebook 07–09 và demo Single/Multi

0.3 Nguyên tắc thực nghiệm

Dữ liệu train dùng để fit; validation dùng chọn checkpoint hoặc threshold; test dùng để báo cáo. Các chương chuỗi giữ thứ tự thời gian. Riêng thí nghiệm graph mới sử dụng giao thức transductive và công khai việc dùng thuộc tính của nút test trong cấu trúc đồ thị; nhãn test không đi vào loss. Không trộn số đo từ các giao thức khác nhau để kết luận kiến trúc tốt nhất.

Quyển báo cáo sử dụng bảng đơn sắc, code căn trái và ảnh notebook có output. Kết quả kế thừa không được huấn luyện lại chỉ để tăng điểm. Phần mới ghi rõ file kết quả và cách chạy. Các giới hạn được đặt cùng thí nghiệm để người đọc hiểu điều gì đã chứng minh và điều gì còn là hướng phát triển.

CHƯƠNG 1. LỊCH SỬ PHÁT TRIỂN AI

1.1. Nền tảng

AI không bắt đầu từ một thư viện hoặc một loại mạng duy nhất. Các nền tảng liên quan đến logic, lý thuyết tính toán, xác suất, tối ưu và mô hình hóa neuron đã hình thành những cách khác nhau để mô tả việc giải quyết vấn đề bằng máy. Một nhánh nhấn mạnh ký hiệu và luật suy luận; một nhánh học quan hệ từ dữ liệu. Hai hướng có thể kết hợp trong một hệ thống thay vì thay thế hoàn toàn nhau.

Khái niệm tính toán hiệu dụng đặt câu hỏi về những thủ tục có thể được thực hiện bằng một chuỗi quy tắc xác định. Khi máy tính có thể biểu diễn dữ liệu và chương trình trong bộ nhớ, các hoạt động như tìm kiếm, kiểm tra điều kiện và tối ưu trở thành đối tượng lập trình. Tuy nhiên, có thể tính được về nguyên tắc không đồng nghĩa có thể tính nhanh với tài nguyên hữu hạn. Đây là khác biệt xuyên suốt lịch sử AI.

Các mô hình neuron ban đầu đơn giản hóa hoạt động thần kinh để nghiên cứu phép tính. Mô hình nhân tạo hiện đại cũng chỉ mượn một số ý tưởng trừu tượng, không phải mô phỏng đầy đủ não sinh học. Trong tiểu luận, một neuron là phép tổ hợp tuyến tính và activation; các hành vi phức tạp xuất hiện từ việc ghép các phép biến đổi cùng quá trình học tham số.

Bài học cho thực nghiệm là phải nêu rõ mô hình toán học, chi phí tính toán và miền dữ liệu. Một mạng có thể biểu diễn nhiều hàm nhưng vẫn khó học do thiếu dữ liệu hoặc gradient không ổn định. Ngược lại, mô hình đơn giản có thể đáp ứng một mục tiêu cụ thể nếu đặc trưng đã chứa thông tin phù hợp.

Góc nhìn	Nội dung	Liên hệ thực nghiệm
Logic	Biểu diễn và suy luận	Quy tắc kiểm tra dữ liệu đầu vào
Thống kê	Ước lượng từ mẫu	Split và metric ngoài mẫu
Tối ưu	Điều chỉnh tham số	Loss, gradient và SGD
Kỹ thuật	Tài nguyên và vận hành	CPU, checkpoint và API

1.2. Turing và Dartmouth

Năm 1950, Alan Turing công bố “Computing Machinery and Intelligence” trên Mind. Thay vì chỉ tranh luận một định nghĩa trừu tượng về suy nghĩ, bài báo xem xét một phép thử dựa trên tương tác và hành vi quan sát được . Cách đặt vấn đề này góp phần thúc đẩy việc chuyển câu hỏi triết học sang một tình huống có thể thảo luận bằng tiêu chí cụ thể.

Phép thử hành vi không tự trả lời mọi câu hỏi về hiểu biết, ý thức hoặc khả năng tổng quát hóa. Một hệ thống có thể thể hiện tốt trong một kịch bản nhưng thất bại ở đầu vào khác. Liên hệ với học máy hiện đại, score trên một tập test chỉ là bằng chứng trong một phân bố và protocol xác định. Nó không phải chứng minh rằng mô hình hiểu thế giới theo cùng cách con người.

Đề xuất Dartmouth được viết năm 1955 bởi John McCarthy, Marvin Minsky, Nathaniel Rochester và Claude Shannon, hướng tới một dự án mùa hè năm 1956. Văn bản nêu tên artificial intelligence và đề cập ngôn ngữ, khái niệm, neuron networks, tính toán và khả năng cải thiện hệ thống . Cần phân biệt năm soạn đề xuất với năm tổ chức hoạt động.

Từ góc nhìn tổ chức khoa học, việc tập hợp những câu hỏi dưới một tên lĩnh vực tạo điều kiện trao đổi phương pháp và xây dựng cộng đồng. Tuy nhiên, một chương trình nghiên cứu rộng không thể được đánh giá như một sản phẩm đã hoàn tất. Bài học tương ứng trong tiểu luận là phân biệt mục tiêu dự kiến, công việc đã thực hiện và phần còn chưa được xác minh.

Mốc	Tài liệu	Ý nghĩa được dùng trong bài
1950	Turing, Mind	Vai trò của tiêu chí đánh giá quan sát được
1955	Đề xuất Dartmouth	Tên AI và chương trình nghiên cứu
1956	Dự án mùa hè Dartmouth	Mốc hình thành cộng đồng nghiên cứu

1.3. AI biểu tượng, hệ chuyên gia và giới hạn

AI biểu tượng biểu diễn đối tượng, quan hệ và luật dưới dạng cấu trúc rời rạc. Khi đã có mô tả phù hợp, máy có thể tìm kiếm trong không gian trạng thái hoặc áp dụng các quy tắc suy luận để đưa ra kết luận. Điểm mạnh là tri thức và đường suy luận có thể được kiểm tra rõ hơn so với một vector trọng số lớn. Khó khăn xuất hiện khi thế giới có nhiều ngoại lệ hoặc thông tin không đầy đủ.

Hệ chuyên gia tập trung vào một miền hẹp, tách cơ sở tri thức và cơ chế suy luận. Việc xây dựng luật cần chuyên gia và công sức cập nhật. Khi số điều kiện tăng, các luật có thể xung đột hoặc khó bao phủ hết tình huống. Dữ liệu ngôn ngữ và ảnh đặc biệt khó mô tả bằng danh sách quy tắc thủ công hoàn chỉnh.

Các giai đoạn giảm kỳ vọng và đầu tư thường được gọi chung là AI winter. Không nên xem đó là một thời điểm duy nhất áp dụng đồng đều cho mọi quốc gia và mọi nhánh nghiên cứu. Những hạn chế về tài nguyên, dữ liệu, độ tin cậy và khoảng cách giữa lời hứa với hệ thống thực góp phần làm kỳ vọng thay đổi. Tiểu luận không sử dụng một chuỗi năm tuyệt đối cho tất cả các giai đoạn suy giảm.

Trong các ứng dụng của bài, luật vẫn giữ vai trò hữu ích: kiểm tra Date tăng dần, giá dương, số tuần đúng hoặc loại file ảnh hợp lệ. Sau lớp kiểm tra này mới tới model học từ dữ liệu. Sự kết hợp cho thấy phát triển hệ thống không chỉ là thay mọi logic bằng một mạng neural.

Ví dụ luật trong web	Mục đích
High không thấp hơn Open/Close	Phát hiện chuỗi OHLC bất hợp lý
Số tuần khách hàng bằng 8	Đảm bảo tensor shape đúng
Ảnh có số pixel giới hạn	Kiểm soát chi phí giải mã
Tên framework thuộc danh sách	Chọn checkpoint đã được đóng gói

1.4. Từ đặc trưng thủ công đến học biểu diễn

Machine learning mô tả việc ước lượng quy luật từ các ví dụ thay vì viết đầy đủ quy tắc cho mọi trường hợp. Trong học có giám sát, thuật toán tối ưu một hàm mất mát từ cặp input - target. Tính hữu ích không nằm ở việc tái tạo tốt tập train, mà ở khả năng tạo dự đoán phù hợp cho dữ liệu chưa dùng để fit. Vì vậy, thiết kế đánh giá là một phần của phương pháp chứ không phải thao tác cuối mang tính hình thức.

Các mô hình tuyến tính tạo quan hệ đơn giản và có chi phí thấp. Cây quyết định biểu diễn các nhánh điều kiện học được. SVM, phương pháp láng giềng và ensemble bổ sung những giả định khác. Không có quy tắc rằng mạng sâu luôn vượt mọi phương pháp cổ điển, đặc biệt với bảng dữ liệu vừa phải và đặc trưng có ý nghĩa rõ.

Backpropagation cho phép tính gradient qua hợp nhiều hàm. Công trình của Rumelhart, Hinton và Williams năm 1986 là một mốc có ảnh hưởng trong việc phổ biến phương pháp học biểu diễn bằng lan truyền ngược . Đạo hàm được tích lũy qua các lớp để cập nhật trọng số; chất lượng biểu diễn vẫn phụ thuộc dữ liệu, loss và tối ưu.

Trong tiểu luận, bản scratch tái hiện nguyên tắc dây chuyền bằng phép nhân ma trận và đạo hàm activation. Nếu gradient sai thì mọi bảng metric sau đó đều có thể gây hiểu nhầm. Việc đối chiếu finite differences có vai trò như một thí nghiệm nhỏ độc lập trước khi đưa mô hình vào các dataset lớn hơn.

```
z = x @ W + b
h = np.maximum(z, 0)
logits = h @ V + c
dV = h.T @ dlogits
dz = (dlogits @ V.T) * (z > 0)
dW = x.T @ dz
```

Đoạn minh họa dùng quy ước sample theo hàng. Hướng của ma trận và phép transpose phải nhất quán với code; cùng một công thức có thể được viết khác khi đổi quy ước vector cột.

1.5. CNN, RNN và attention trong dòng phát triển

CNN khai thác cấu trúc không gian bằng kết nối cục bộ và chia sẻ kernel. Thành công của AlexNet trong cuộc thi ImageNet năm 2012 gắn với một hệ thống gồm kiến trúc, dữ liệu lớn, GPU và các lựa chọn tối ưu, không chỉ một thay đổi đơn lẻ . Tiểu luận dùng CNN nhỏ phục vụ kiểm chứng; không gọi kiến trúc một lớp tích chập là AlexNet.

RNN duy trì trạng thái theo bước thời gian. LSTM bổ sung cơ chế cổng và cell state để cải thiện việc lưu giữ tín hiệu qua chuỗi; công trình của Hochreiter và Schmidhuber năm 1997 là tài liệu nền tảng . Các biến thể được phát triển để xử lý hạn chế của mạng hồi tiếp đơn giản, nhưng vẫn cần quy trình chia thời gian đúng và target được định nghĩa rõ.

Attention cho phép kết hợp thông tin bằng trọng số phụ thuộc nội dung. Transformer trong "Attention Is All You Need" năm 2017 sử dụng attention làm thành phần trung tâm thay cho recurrence trong kiến trúc được đề xuất . Điều này không khiến CNN hoặc RNN mất giá trị trong mọi nhiệm vụ; lựa chọn thực tế còn liên quan đến quy mô, độ trễ và dữ liệu.

Ba dataset ở các trang tiếp theo giúp liên hệ lịch sử học biểu diễn với thực nghiệm: CDC cho thấy giá trị của đặc trưng bảng; MNIST thể hiện lợi ích của cấu trúc ảnh; AAPL cho thấy thứ tự thời gian và baseline có vai trò quyết định. Đây là minh họa hiện đại, không phải tái tạo nguyên vẹn thí nghiệm của những công trình lịch sử.

Họ mô hình	Giả định nổi bật	Thực nghiệm trong tiểu luận
MLP	Tương tác phi tuyến giữa các cột	CDC, giá nhà
CNN	Tính cục bộ và kernel dùng chung	MNIST, EuroSAT
RNN	Trạng thái cập nhật theo thứ tự	AAPL, mua lại
Transformer	Attention giữa các vị trí	Chỉ giới thiệu lý thuyết

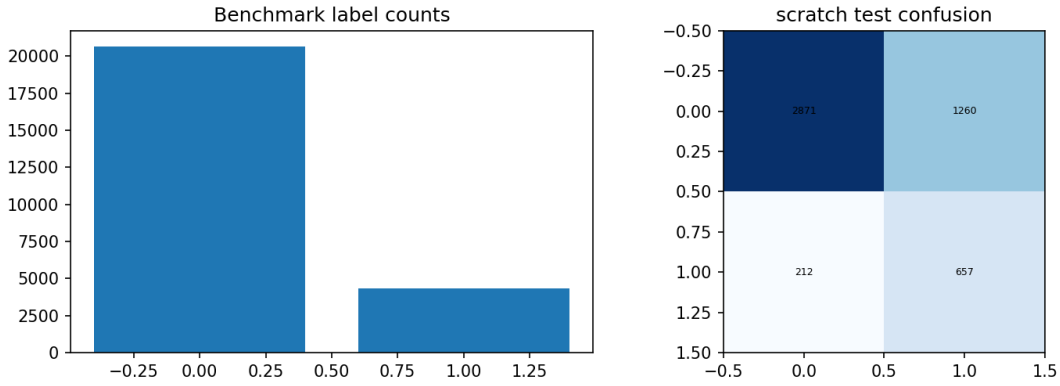
1.6. Dataset minh họa: CDC Diabetes

Nguồn: https://www.kaggle.com/datasets/alexteboul/diabetes-health-indicators-dataset

Dataset	Nguồn gốc	Train / Val / Test	Input
CDC Diabetes	253,680	16000 / 4000 / 5000	21

Mẫu	Input chuẩn hóa (tối đa 8 số đầu)	Target gốc
20000	1.0970, 1.1350, 0.1980, -0.1110, 1.0700, -0.2150, -0.3420, -1.6420	0.0000

Mẫu	Input chuẩn hóa (tối đa 8 số đầu)	Target gốc
20001	-0.9110, -0.8810, 0.1980, -1.1350, -0.9340, -0.2150, -0.3420, -1.6420	0.0000
20002	1.0970, -0.8810, 0.1980, 0.6200, -0.9340, -0.2150, -0.3420, -1.6420	1.0000



Hình kế thừa 1 Phân bố/đánh giá trên phần benchmark sử dụng; hình được tạo từ dữ liệu thực.

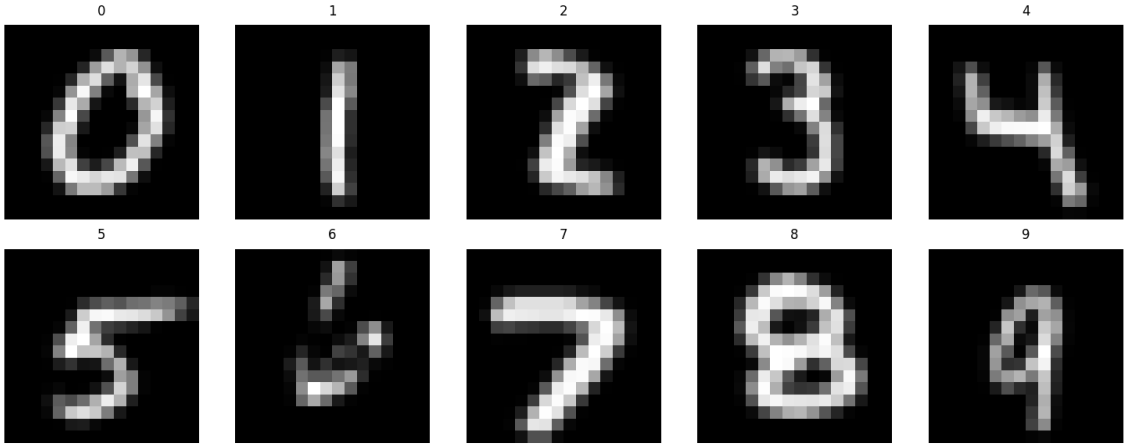
CDC minh họa dữ liệu bảng có các chỉ báo được định nghĩa trước. Bài gộp nhãn tiền tiểu đường và tiểu đường thành lớp dương, nên không gọi score này là xác suất chẩn đoán tiểu đường đơn thuần. Các nhóm predictor giống nhau không xuất hiện đồng thời ở train và test. Dataset không đại diện tự động cho một bệnh viện hoặc dân số khác. Phân bố benchmark phản ánh tập con đã chọn, không phải số bệnh nhân độc lập ngoài thực tế.

1.7. Dataset minh họa: MNIST

Nguồn: https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz

Dataset	Nguồn gốc	Train / Val / Test	Input
MNIST	70,000	10000 / 2000 / 2000	16 × 16 × 1

Mẫu	Input chuẩn hóa (tối đa 8 số đầu)	Target gốc
12000	-0.4980, -0.4980, -0.4980, -0.4980, -0.4980, -0.4980, -0.4980, -0.4980	2.0000
12001	-0.4980, -0.4980, -0.4980, -0.4980, -0.4980, -0.4980, -0.4980, -0.4980	3.0000
12002	-0.4980, -0.4980, -0.4980, -0.4980, -0.4980, -0.4980, -0.4980, -0.4980	0.0000



Hình kế thừa 2 Một ảnh cho mỗi lớp MNIST trong benchmark; resize 16.

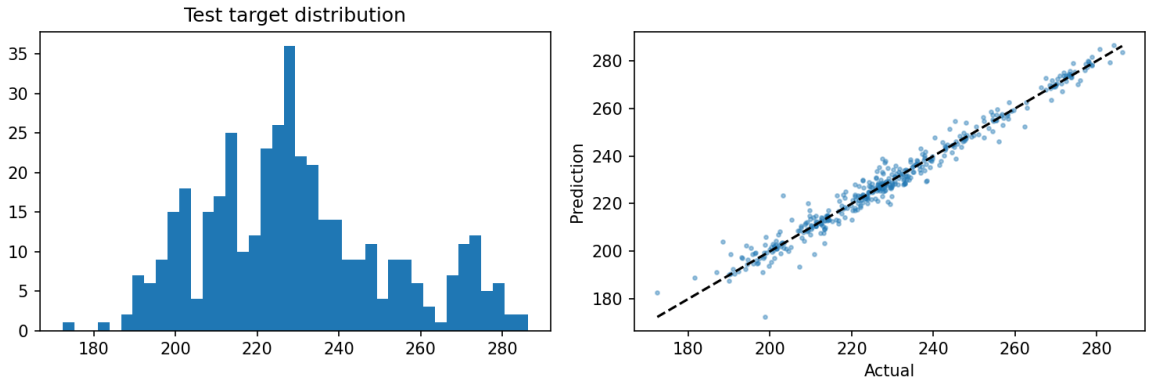
MNIST có nền và bố cục tương đối chuẩn hóa, thích hợp kiểm tra công thức CNN trên máy cá nhân. Ảnh tải từ điện thoại hoặc nhiều ký tự trên cùng một ảnh có phân bố khác. Hình minh họa gắn nhãn thật; các giá trị input trong bảng là phần đầu tensor chuẩn hóa, không phải code định danh lớp. Kết quả ở Chương 3 phải đọc cùng kích thước 16 × 16 và số ảnh benchmark đã nêu.

1.8. Dataset minh họa: AAPL

Nguồn: https://github.com/tuananhtrieu1305/ISD_Assignment01/tree/main/A06/datasets

Dataset	Nguồn gốc	Train / Val / Test	Input
AAPL	2,766	1880 / 403 / 404	30 × 8

Mẫu	Input chuẩn hóa (tối đa 8 số đầu)	Target gốc
2283	-0.6450, -0.4360, -0.6810, -0.4830, -0.6700, -1.0540, -0.3820, -2.0280	-0.0076
2284	2.2240, 0.2520, 1.7330, 2.7670, 0.2690, -0.4770, -0.0280, -0.6080	-0.0213
2285	0.4160, -0.3820, 0.2640, 0.8770, 0.4460, -0.3490, -0.0350, -0.3490	0.0165



Hình kế thừa 3 Phân bố/đánh giá trên phần benchmark sử dụng; hình được tạo từ dữ liệu thực.

AAPL cho thấy việc giữ thứ tự thời gian quan trọng hơn xáo trộn ngẫu nhiên để tăng vẻ đồng nhất giữa các tập. Các hàng input là lịch sử, output là log-return phiên kế tiếp. Quan sát phân bố giá không đủ kết luận model có lợi thế dự báo. Bài học xuyên suốt lịch sử AI là điều chỉnh kỳ vọng theo dữ liệu và protocol kiểm chứng, thay vì chỉ nhìn tên của một kiến trúc mới.

CHƯƠNG 2. ML CƠ BẢN

2.1. Học có giám sát

Học có giám sát tìm một hàm f từ input x tới target y dựa trên các cặp đã quan sát. Classification trả lớp hoặc score từng lớp; regression trả một đại lượng liên tục. Trong bài, CDC là classification hai lớp và giá nhà là regression. Hai nhiệm vụ cần loss và metric khác nhau, dù có thể dùng cùng bộ khung MLP.

Tập train được dùng cập nhật tham số. Validation phục vụ chọn epoch hoặc cấu hình trong phạm vi đã định. Test mô phỏng dữ liệu chưa dùng cho hai bước trước. Nếu liên tục xem test rồi sửa model, test đã trở thành thông tin thiết kế và không còn vai trò độc lập ban đầu. Tiểu luận lưu checkpoint theo validation loss và ghi tiêu chí lựa chọn implementation trước bảng test.

Tính khái quát hóa phụ thuộc vào cách mẫu được hình thành. Các dòng giống nhau, cùng địa chỉ hoặc cùng khách hàng có thể liên hệ với nhau. Chia ngẫu nhiên từng dòng không luôn thích hợp. CDC dùng nhóm predictor, giá nhà dùng nhóm Address. Với chuỗi thời gian ở Chương 4, thứ tự nhãn được ưu tiên thay cho chia ngẫu nhiên.

Đánh giá ngoài mẫu không chỉ là chọn một hàm metric. Cần giải thích đơn vị output, định nghĩa lớp dương và phạm vi dữ liệu. Accuracy lớn có thể xuất hiện khi luôn đoán lớp phổ biến; RMSE lớn hoặc nhỏ cũng phải được đặt cạnh phân bố giá và baseline. Vì vậy, mỗi dataset có cả bảng metric và nhận xét về dữ liệu.

Dataset	Nguồn gốc	Train / Val / Test	Input
CDC Diabetes	253,680	16000 / 4000 / 5000	21
Giá nhà Việt Nam	30,229	19600 / 4665 / 5964	6

2.2. Các phương pháp tuyến tính và dựa trên khoảng cách

Hồi quy tuyến tính mô hình hóa y bằng xW+b. Khi cần hạn chế hệ số quá lớn, Ridge thêm hình phạt bình phương trọng số. Cách này tạo một baseline dễ kiểm tra, nhưng biểu diễn chỉ tuyến tính trong các đặc trưng đã cung cấp. Biến đổi log hoặc thêm biến tương tác có thể thay đổi miền hàm mà mô hình tuyến tính biểu diễn.

Logistic regression ánh xạ tổ hợp tuyến tính sang score nhị phân hoặc softmax đa lớp và tối ưu cross-entropy. Dù tên có chữ regression, ứng dụng thường gặp là phân loại. Ngưỡng chuyển score sang nhãn và class weight có ảnh hưởng tới Precision / Recall. Bài dùng LogisticRegression có class_weight balanced làm baseline cho CDC; điều này không đồng nghĩa score đã calibration.

K-nearest neighbors dự đoán theo các mẫu gần trong không gian đặc trưng. Khi các cột khác đơn vị, khoảng cách có thể bị một cột chi phối, nên chuẩn hóa và lựa chọn metric quan trọng. SVM tìm biên phân tách và có thể dùng kernel để biểu diễn phi tuyến. Hai phương pháp này được giới thiệu để đối chiếu nguyên lý, không tính vào các lần train được báo cáo.

Không nên chọn thuật toán chỉ vì nó phức tạp hơn. Một baseline có ít giả định, dễ debug giúp phát hiện lỗi xử lý hoặc đánh giá. Trong thực nghiệm giá nhà, target được biến đổi log 1 p và tất cả metric cuối được đổi về đơn vị gốc; nếu so một bảng loss log với một bảng RMSE tỷ đồng thì đó là hai đại lượng khác nhau.

Phương pháp	Cơ chế	Điểm cần kiểm soát
Linear/ Ridge	Ánh xạ tuyến tính	Thang đo, regularization, target transform
Logistic	Logit tuyến tính, cross-entropy	Lớp dương, weight, threshold
kNN	Láng giềng trong feature space	Khoảng cách và chuẩn hóa
SVM	Biên phân tách / kernel	C, kernel và chi phí dữ liệu lớn

2.3. Cây, ensemble và mạng MLP

Decision tree chia không gian đặc trưng thành các vùng bằng điều kiện theo cột. Cây sâu có thể mô tả tương tác phức tạp nhưng dễ bám nhiễu. Random forest kết hợp nhiều cây được huấn luyện với ngẫu nhiên hóa để giảm phương sai so với một cây riêng lẻ. Boosting xây dựng mô hình theo chuỗi nhằm giảm phần lỗi còn lại. Các họ ensemble có khác biệt về tối ưu và rủi ro overfit; không gọi mọi ensemble là cùng một thuật toán.

MLP gồm các lớp affine xen activation. Nếu chỉ ghép lớp tuyến tính không có phi tuyến, toàn bộ mạng vẫn biểu diễn một ánh xạ tuyến tính. ReLU đưa điểm gãy vào hàm và làm gradient đơn giản ở các vị trí khác 0. Tiểu luận dùng một hidden layer 32 neuron để cân bằng khả năng biểu diễn và khả năng đối chiếu code.

Output của CDC có hai logits; softmax biến logits thành score tổng bằng 1. Output giá nhà là một số chuẩn hóa, không qua sigmoid. Loss classifier là weighted cross-entropy; loss regression là mean squared error. Scratch, Keras và PyTorch dùng cùng các công thức này thay vì lựa chọn mặc định khác nhau.

Baseline random forest trong bài có 100 cây và max_depth 12, seed 42. Đây là cấu hình cố định, chưa được tìm kiếm tối ưu. Bảng baseline được trình bày riêng với ba implementation MLP, vì một random forest không phải bản thay thế tương đương của cùng kiến trúc neural.

```
hidden = relu(x @ W + b)          # (B, D) -> (B, 32)
logits = hidden @ V + c           # -> (B, 2) hoặc (B, 1)
loss = weighted_cross_entropy(logits, y)  # classification
# Regression: mean((logits[:, 0] - y_scaled)**2)
```

2.4. Dữ liệu và tiền xử lý: CDC Diabetes

Nguồn: https://www.kaggle.com/datasets/alexteboul/diabetes-health-indicators-dataset

Dataset	Nguồn gốc	Train / Val / Test	Input
CDC Diabetes	253,680	16000 / 4000 / 5000	21

Mẫu	Input chuẩn hóa (tối đa 8 số đầu)	Target gốc
20000	1.0970, 1.1350, 0.1980, -0.1110, 1.0700, -0.2150, -0.3420, -1.6420	0.0000
20001	-0.9110, -0.8810, 0.1980, -1.1350, -0.9340, -0.2150, -0.3420, -1.6420	0.0000
20002	1.0970, -0.8810, 0.1980, 0.6200, -0.9340, -0.2150, -0.3420, -1.6420	1.0000

Nguồn CDC có nhãn Diabetes_012 và 21 biến khảo sát. Nhãn 0 được giữ là không có tiểu đường; nhãn 1 và 2 được gộp thành nhóm tiền tiểu đường/tiểu đường. Đây là lựa chọn target của bài, không phải một nhãn lâm sàng mới. Các bản ghi trùng được loại trước khi nhóm các vector predictor giống nhau.

Sau group split, mỗi tập lấy một benchmark có giữ tỷ lệ nhãn: 16.000 train, 4.000 validation và 5.000 test. Việc lấy tập con được thực hiện độc lập bên trong từng split, vì vậy không đưa nhóm đã tách sang tập khác. Mean/std fit từ train; class weights bằng N/(K×count_k) trên train.

Các mã như Age, Education hoặc Income trong khảo sát là nhóm phân loại có thứ tự; không diễn giải trực tiếp Age =9 là chín tuổi. Bài dùng chúng ở dạng số theo nguồn để tạo baseline MLP, và ghi nhận giới hạn của việc xem khoảng cách giữa mã là đồng đều.

Tập	Lớp 0	Lớp 1
train	13227	2773
val	3319	681
test	4131	869

2.5. Dữ liệu và tiền xử lý: Giá nhà Việt Nam

Nguồn: https://www.kaggle.com/datasets/nguyentiennhan/vietnam-housing-dataset-2024

Dataset	Nguồn gốc	Train / Val / Test	Input

Dataset	Nguồn gốc	Train / Val / Test	Input
Giá nhà Việt Nam	30,229	19600 / 4665 / 5964	6

Mẫu	Input chuẩn hóa (tối đa 8 số đầu)	Target gốc
7	0.4550, 0.1820, 2.2440, 1.1680, 0.5890, 1.3270	9.9000
16	3.2220, 4.0600, 3.5090, -0.1440, 1.875, -0.0870	9.4000
33	0.2890, -0.1620, -0.0450, -0.1440, 0.5890, 1.8640	6.5000

File giá nhà có 30.229 dòng và 12 cột. Thực nghiệm giữ sáu predictor số: Area, Frontage, Access Road, Floors, Bedrooms, Bathrooms. Các giá trị thiếu được điền bằng median train, các giá trị âm ở predictor coi là thiếu. Chỉ giữ hàng Area và Price dương; không cắt bỏ ngoại lệ theo phân vị của toàn bộ dữ liệu.

Address được dùng làm khóa group split để những tin có cùng địa chỉ chính xác không nằm ở nhiều tập. Nó không được đưa vào predictor. Điều này giúp giảm một dạng leakage nhưng làm mạng thiếu thông tin vị trí quan trọng; cùng diện tích và số phòng có thể có giá khác nhau giữa các địa bàn.

Input dùng log 1 p rồi chuẩn hóa train. Target dùng log 1 p(Price), chuẩn hóa train và đổi ngược khi báo cáo tỷ VND. Biến đổi làm thay đổi trọng số tương đối của sai số khi tối ưu: giảm loss trong miền log không bảo đảm giảm mạnh lỗi tuyệt đối của nhà rất đắt. Chưa xác minh từng tin rao là một giao dịch đã hoàn tất.

2.6. Cài đặt MLP scratch và đạo hàm

```
z = x @ p['w'] + p['b']
h = np.maximum(z, 0)
logits = h @ p['v'] + p['c']
# dz_out đã chứa hệ số trung bình batch và class weight.
grad_v = h.T @ dz_out
grad_c = dz_out.sum(axis=0)
grad_hidden = dz_out @ p['v'].T
grad_z = grad_hidden * (z > 0)
grad_w = x.T @ grad_z
grad_b = grad_z.sum(axis=0)
```

Cross-entropy được tính từ logits trừ max theo hàng trước khi lấy exp. Bước dịch này không đổi softmax nhưng giảm nguy cơ tràn số. Gradient logits bằng softmax trừ one-hot, nhân weight của nhãn thật và chia batch size. Loss weighted được lấy trung bình theo số mẫu, thống nhất với code hai framework; không chia thêm theo tổng weight.

Regression có gradient 2(pred- target)/B. Từ gradient output, quy tắc dây chuyền đi ngược qua head, ReLU và dense đầu. Bias gradient là tổng theo trục batch. Do hidden layer có kích thước nhỏ, toàn bộ phép tính có thể biểu diễn bằng NumPy, giúp đọc rõ từng tensor.

Trước cập nhật, norm chung được tính trên mọi gradient; nếu lớn hơn 1 thì cùng scale tất cả gradient. SGD cập nhật p←p− lr ×g với lr 0,02. Không có momentum hoặc Adam trong phép so sánh chính. Sự đơn giản của optimizer giúp giảm số khác biệt không cần thiết khi kiểm tra giữa framework.

Mã đầy đủ: src/scratch.py; notebook 02 chứa cả định nghĩa forward, objective, backward và step.

2.7. MLP Keras và PyTorch tương ứng

```
# Keras: gán trọng số giống NumPy trước train.
inputs = tf.keras.layers.Input(shape=(features,))
h = tf.keras.layers.Dense(32, activation='relu')(inputs)
outputs = tf.keras.layers.Dense(n_outputs)(h)
model = tf.keras.Model(inputs, outputs)

# PyTorch: giữ cùng hướng ma trận (D, H).
def forward(self, x):
    p = self.params
    h = torch.relu(x @ p['w'] + p['b'])
    return h @ p['v'] + p['c']
```

PyTorch ParameterDict lưu các tensor có thể học theo đúng shape của NumPy. Những phép toán @, cộng bias và ReLU được autograd theo dõi. Keras dùng các layer Dense chuẩn, gán kernel và bias từ cùng bộ initialization. Cả hai mô hình nhận batch có cùng thứ tự mẫu và cùng dtype float 32 trong huấn luyện.

Vòng train Keras dùng GradientTape và SGD với global_ clipnorm 1. PyTorch gọi loss. backward, clip_grad_norm_ và optimizer.step. Scratch tự tính g. Sự khác biệt chính cần khảo sát là đường thực thi và độ chính xác số học, không phải độ rộng hoặc activation. Thời gian đo bao gồm overhead thực thi nên không đại diện một benchmark GPU tối ưu.

Với seed 42 và phép cập nhật gần như tương đương, ba metric có thể bằng nhau đến nhiều chữ số. Đây không phải bằng chứng số liệu được sao chép: mỗi implementation được train độc lập, có history và native checkpoint riêng. Bài lưu gradient checks và dự đoán test để người đọc có thể kiểm tra lại.

Dataset / cài đặt	Tham số	Epoch tốt	Train giây
CDC Diabetes / scratch	770	18	0.4625
CDC Diabetes / keras	770	18	2.2851
CDC Diabetes / pytorch	770	18	2.0878

2.8. Kết quả và baseline: CDC Diabetes

Cài đặt	Accuracy	Macro-F1	F1	AP

Cài đặt	Accuracy	Macro-F1	F1	AP
scratch	0.7056	0.6338	0.4716	0.4432
keras	0.7056	0.6338	0.4716	0.4432
pytorch	0.7056	0.6338	0.4716	0.4432

Chọn trên validation bằng Macro-F1: scratch. Baseline đơn giản trên test: Accuracy =0.8262, Macro-F1 =0.4524.

Mô hình	Accuracy	Macro-F1	F1	AP
LogisticRegression	0.7146	0.6403	0.4767	0.4362
RandomForest	0.7890	0.6731	0.4785	0.4497
MLP	0.7056	0.6338	0.4716	0.4432

Accuracy và F1 được đọc cùng nhau vì lớp dương ít hơn. Class weighting tăng mức phạt lớp dương, có thể tăng Recall nhưng làm Precision giảm. Vì vậy, không chọn winner bằng Accuracy duy nhất hoặc đổi threshold sau khi xem test. Tiểu luận dùng argmax hai logits, tương đương ngưỡng 0,5 của softmax lớp dương.

Baseline LogisticRegression và RandomForest có cơ chế biểu diễn khác MLP. Nếu một baseline tốt hơn ở metric cụ thể, phải ghi nhận kết quả đó thay vì suy luận mô hình neural mặc định vượt trội. Các cấu hình chưa qua tìm kiếm nên kết luận chỉ nằm trong phạm vi đã chạy.

2.9. Kết quả và baseline: Giá nhà Việt Nam

Cài đặt	RMSE	MAE	R^2
scratch	1.8760	1.4980	0.2799
keras	1.8760	1.4980	0.2799
pytorch	1.8760	1.4980	0.2799

Chọn trên validation bằng RMSE: keras. Baseline đơn giản trên test: RMSE=2.2138, MAE=1.8356, R^2=-0.0028.

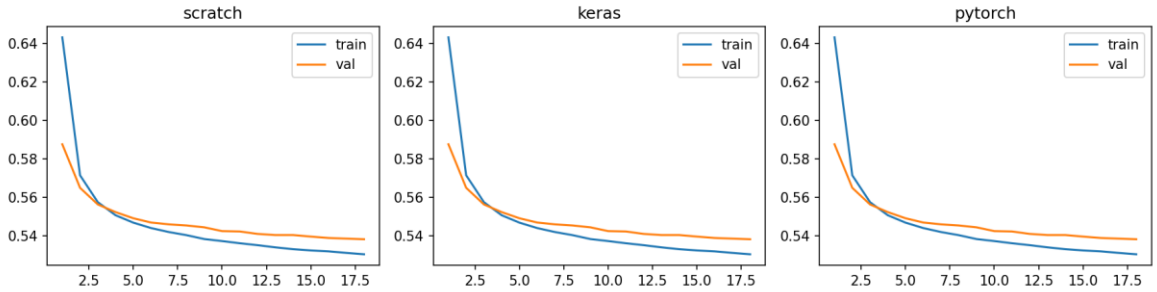
Mô hình	RMSE	MAE	R^2

Mô hình	RMSE	MAE	R^2
Ridge	1.9465	1.5550	0.2247
RandomForest	1.8542	1.4719	0.2965
MLP	1.8760	1.4980	0.2799

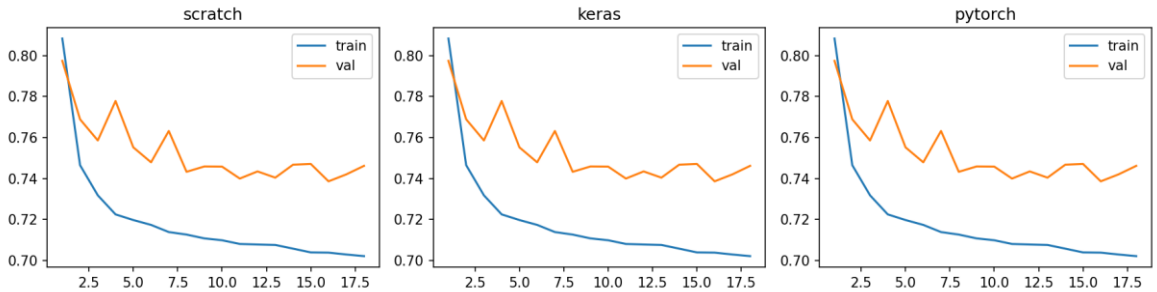
RMSE nhạy với sai số lớn còn MAE thể hiện lỗi tuyệt đối trung bình. R^2 được tính trong đơn vị giá gốc, không phải trên target chuẩn hóa. Cần so với baseline dự đoán mean train và hai mô hình cổ điển. Một MLP trên sáu cột số chưa nắm được vị trí, tình trạng pháp lý và nhiều yếu tố của bất động sản.

Các dự đoán là ước lượng từ dữ liệu rao bán của nguồn, không phải định giá bảo đảm. Chênh lệch rất nhỏ giữa ba implementation không có ý nghĩa thực tiễn lớn so với độ thiếu thông tin của predictor. Để cải thiện cần đánh giá thêm feature và protocol, không chỉ đổi framework.

2.10. Learning curve của hai bài ML



Hình kế thừa 4 Train/validation loss: CDC Diabetes.



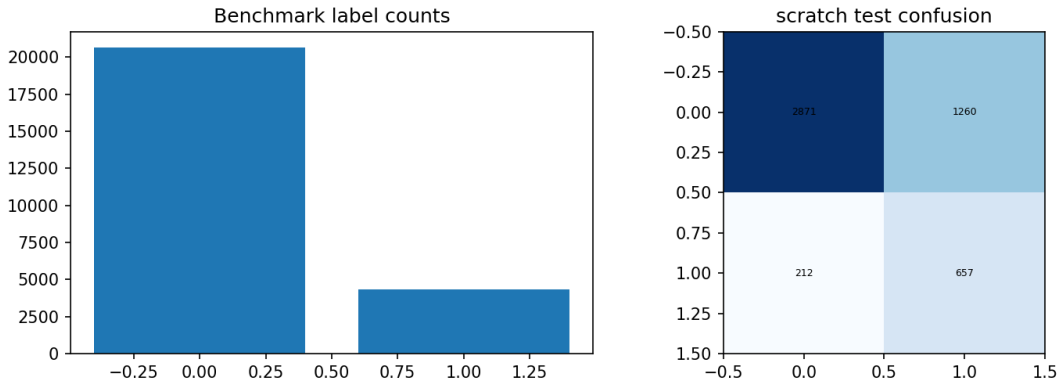
Hình kế thừa 5 Train/validation loss: Giá nhà Việt Nam.

Mỗi đường biểu diễn loss theo epoch; train loss được tích lũy trong lúc batch cập nhật, còn validation loss tính từ model cuối epoch. Mức loss của classification và regression không có cùng ý nghĩa hay đơn vị, nên không so cao thấp giữa hai hình. Epoch tốt được chọn riêng cho từng implementation bằng validation loss nhỏ nhất.

Đường học gần nhau hỗ trợ tính nhất quán của code. Tuy nhiên, sự trùng khớp không chứng minh dữ liệu có đủ thông tin hoặc việc chia tập hoàn hảo. Kiểm tra thuật toán và đánh giá tính

đại diện dữ liệu là hai lớp kiểm chứng khác nhau. Một triển khai đúng có thể vẫn dự đoán yếu.

2.11. Phân bố, ví dụ sai và chi phí



Hình kế thừa 6 CDC: phân bố benchmark và confusion matrix của implementation chọn bằng validation.

Chỉ số	Thực	Dự đoán	Sai số / score
729	9.9500	3.1287	6.8213
6824	10.0000	3.7899	6.2101
16227	9.8000	3.7044	6.0956
6188	9.8000	3.7614	6.0386
20425	10.0000	4.0924	5.9076
9578	1.2000	7.1059	5.9059

Với CDC, confusion matrix có hàng là lớp thật và cột là lớp dự đoán. Với giá nhà, bảng liệt kê các sai số lớn nhất của implementation đã chọn; không dùng những ví dụ này để chọn lại model. Dữ liệu chuẩn hóa trong bảng input mẫu phục vụ tái lập, còn metric regression được đổi ngược về tỷ VND.

Các hàng giá nhà sai lớn gợi ý nhu cầu bổ sung vị trí hoặc phân tích phân khúc giá, nhưng chưa xác định nguyên nhân cho từng căn nhà. Những nhận xét đó là giả thuyết cho nghiên cứu sau, không phải thông tin đã có trong dữ liệu thực nghiệm.

Dataset / cài đặt	Tham số	Epoch tốt	Train giây
Giá nhà Việt Nam / scratch	257	16	0.5001

Dataset / cài đặt	Tham số	Epoch tốt	Train giây
Giá nhà Việt Nam / keras	257	16	2.2980
Giá nhà Việt Nam / pytorch	257	16	2.6011

2.12. Tiểu kết và giới hạn ML

Chương 2 đã trình bày các nguyên lý ML nền tảng, chuẩn bị hai dataset và huấn luyện ba implementation MLP độc lập trên mỗi dataset. Các baseline tuyến tính và random forest bổ sung góc nhìn ngoài neural network. Bản scratch thực hiện thật cả đạo hàm và cập nhật, được kiểm tra trước khi chạy dữ liệu.

Nhóm thực nghiệm này cho thấy việc thống nhất loss reduction, shape và initialization có vai trò lớn khi đối chiếu framework. Khi các điều kiện đó trùng nhau, lựa chọn NumPy, Keras hoặc PyTorch không tự làm thay đổi bài toán thống kê. Framework giúp tổ chức và tối ưu tính toán, còn giá trị dự báo phụ thuộc thông tin đầu vào và protocol.

CDC có nhãn gộp và dữ liệu khảo sát; không được diễn giải thành một công cụ chẩn đoán. Giá nhà dùng sáu predictor số và thiếu vị trí; không đại diện một hệ thống định giá đầy đủ. Group split giảm một số dạng trùng lặp nhưng không bảo đảm tổng quát hóa sang bệnh viện mới, thị trường mới hoặc thời kỳ khác.

Hướng mở rộng hợp lý gồm đánh giá nhiều seed, thêm validation folds ở cấp nhóm, khảo sát đặc trưng và calibration cho classifier. Những bước này cần được thực hiện với test mới hoặc giữ test hiện tại khóa kín. Trong bài hiện tại không có tìm siêu tham số hay lựa chọn threshold bằng test; các giới hạn được giữ để kết luận trung thực.

Đã thực hiện	Chưa thực hiện
Scratch/ Keras / PyTorch cùng MLP	Đánh giá nhiều seed và khoảng tin cậy
Hai dataset có split rõ	Đánh giá chuyển miền ngoài nguồn dữ liệu
Baseline và phân tích lỗi	Calibration hoặc triển khai ra quyết định thực
Checkpoint và web	Bảo đảm chất lượng cho mọi đầu vào mới

CHƯƠNG 3. CNN

3.1. Động cơ và cấu trúc ảnh

Ảnh có cấu trúc lân cận theo hai trục không gian. Nếu trải tất cả pixel thành vector và nối đầy đủ ngay từ đầu, mô hình không được cung cấp trực tiếp giả định rằng các vùng cục bộ có thể chia sẻ kiểu đặc trưng. CNN dùng kernel trên các vị trí để khai thác giả định này, giảm số tham số ở lớp trích đặc trưng so với dense toàn ảnh.

Một kernel nhỏ có thể phản ứng với tương phản hoặc cấu trúc màu đơn giản. Qua nhiều lớp, receptive field tăng và biểu diễn có thể kết hợp vùng lớn hơn. Tuy nhiên, không cần gán mọi kernel học được cho một khái niệm như "mắt" hoặc "cây" nếu chưa phân tích. Tiểu luận dùng một lớp conv để đối chiếu có kiểm soát, không khẳng định mạng học được cấu trúc ngữ nghĩa sâu.

Phép convolution trong các framework thường là cross-correlation: kernel không bị lật như định nghĩa convolution toán học cổ điển. Mã scratch dùng cùng quy ước để so gradient đúng. Translation equivariance của phép tích chập cũng khác với invariance của toàn bộ classifier; pooling, biên ảnh và head có thể thay đổi tính chất này.

Hai dataset được chọn là MNIST và EuroSAT. MNIST có mẫu chữ số đơn giản hơn về bối cảnh; EuroSAT có màu và texture của bề mặt đất. Cùng một CNN nhỏ giúp quan sát tác động của dữ liệu khi giữ kiến trúc cố định, thay vì so hai mạng có quy mô hoàn toàn khác nhau.

Dataset	Nguồn gốc	Train / Val / Test	Input
MNIST	70,000	10000 / 2000 / 2000	16 × 16 × 1
EuroSAT	27,000	6400 / 1600 / 2000	16 × 16 × 3

3.2. Convolution, pooling và số tham số

```
# Input NHWC: (B, 16, 16, C)
# Kernel: (3, 3, C, 8), stride1, valid
# Conv output: (B, 14, 14, 8)
# ReLU: giữ nguyên shape
# AveragePool2: (B, 7, 7, 8)
# Flatten: (B, 392)
# Dense: (B, 10)
```

Convolution tại mỗi vị trí là tổng tích giữa patch và kernel, cộng bias theo kênh output. Với padding valid và stride 1, chiều không gian giảm từ 16 xuống 14. ReLU giữ phần dương và không đổi shape. Average pooling 2 × 2 lấy trung bình bốn giá trị không chồng lấp, đưa shape về 7 × 7. Head dense nhận 392 giá trị và trả 10 logits.

Kernel có 3 × 3 ×C× 8 trọng số cùng 8 bias. Head có 392 × 10 trọng số cùng 10 bias. Do đó, MNIST và EuroSAT khác số tham số convolution vì số kênh 1 và 3; head giống nhau. Số

pixel của feature map không phải số tham số học được: cùng một kernel được dùng tại mọi vị trí.

Tiểu luận chọn average pooling để đạo hàm đơn giản và rõ: gradient từ một ô pooled được chia đều bốn ô input. Max pooling cần lưu vị trí argmax; nó có thể dùng trong mạng khác nhưng không phải phép toán đang chạy ở đây. Việc đổi pooling mà không đổi tên cấu hình sẽ làm hỏng tính công bằng của phép so sánh.

Layer	Tham số MNIST	Tham số EuroSAT
Conv 3 × 3, 8 filters	80	224
Dense 392→10	3930	3930
Tổng	4010	4154

3.3. Loss và lan truyền ngược qua CNN

Classifier nhận logits từ head, sử dụng weighted softmax cross-entropy. Gradient head có dạng giống MLP. Sau khi nhân với ma trận head chuyển vị, gradient được reshape về feature map pooled. Mỗi phần tử được phân phối đều cho bốn vị trí trước average pooling; sau đó nhân mặt nạ ReLU của pre-activation conv.

Gradient kernel được tính bằng tổng outer product giữa patch đầu vào và gradient tại output tương ứng. Việc kernel được dùng ở nhiều vị trí khiến gradient cần cộng trên cả batch và hai trục không gian. Bias gradient là tổng output gradient theo cùng ba trục. Trong kiến trúc một conv này không cần dùng gradient ảnh input để cập nhật một layer trước nó.

Mã NumPy dùng sliding_window_view để biểu diễn patch mà không viết một vòng lặp Python riêng cho từng pixel. Tensordot thực hiện phép co các trục tương ứng. Đây là một cách tăng hiệu quả tính toán, nhưng vẫn là code forward / backward tự triển khai và không gọi autograd. Quy tắc đạo hàm không đổi khi thay vòng lặp bằng phép tensor.

```
dpool = dflat.reshape(B, 7, 7, 8)
dconv = np.repeat(np.repeat(dpool, 2, axis=1), 2, axis=2) / 4
dconv *= (preactivation > 0)
dW = np.tensordot(patches, dconv,
                  axes=([0, 1, 2], [0, 1, 2]))
db = dconv.sum(axis=(0, 1, 2))
```

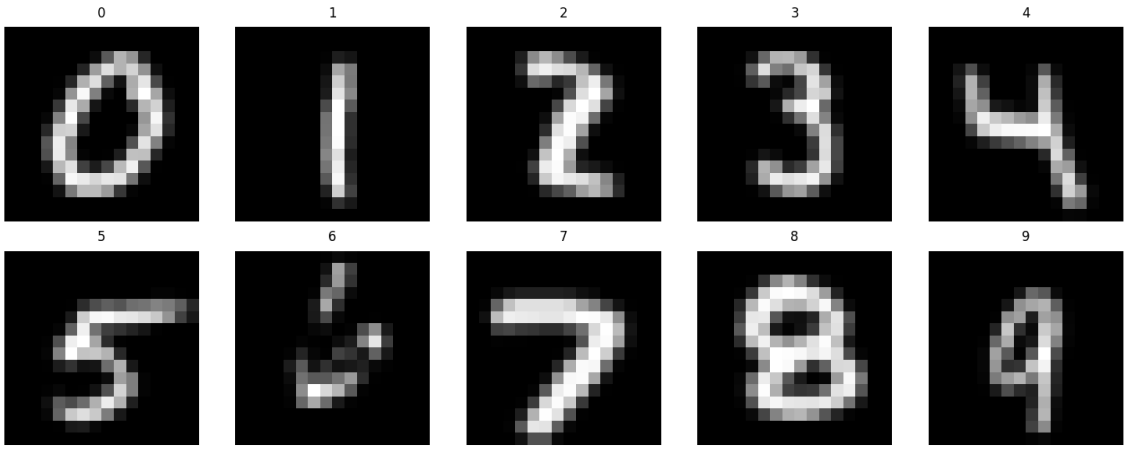
Finite differences kiểm tra một số phần tử của từng tensor parameter; gradient đầy đủ được so với hai autograd trên mini- batch nhỏ. Các kiểm tra không dùng test dataset để tối ưu mô hình.

3.4. Dataset ảnh: MNIST

Nguồn: https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz

Dataset	Nguồn gốc	Train / Val / Test	Input

Dataset	Nguồn gốc	Train / Val / Test	Input
MNIST	70,000	10000 / 2000 / 2000	16 × 16 × 1



Hình kế thừa 7 Mẫu thật của mười lớp; hình từ benchmark, không phải ảnh sinh.

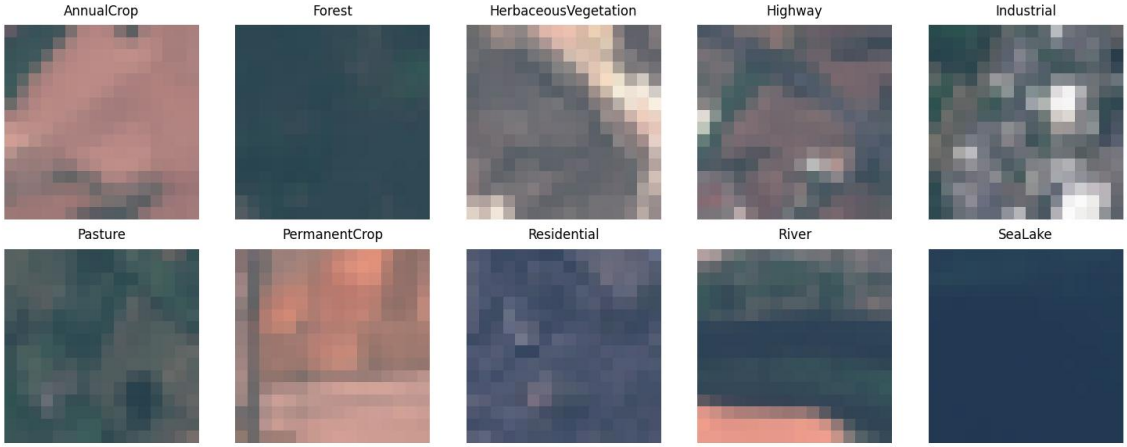
Nguồn MNIST có 60.000 ảnh train và 10.000 ảnh test. Bài chọn 10.000 train cùng 2.000 validation từ phần train chính thức, và 2.000 test từ phần test chính thức, giữ tỷ lệ lớp bằng stratified sampling. Ảnh gốc 28 × 28 grayscale được resize về 16 × 16. Quy mô toàn nguồn và benchmark được phân biệt trong bảng.

Pixel được chia 255 rồi chuẩn hóa bằng mean/std channel trên train. Việc thu nhỏ giúp scratch train nhanh nhưng có thể làm mất nét mảnh. Kết quả không so trực tiếp với mô hình trên full MNIST 28 × 28 hoặc có augmentation. Ảnh nhập web cần gần dạng chữ số sáng trên nền tối; nhiều ký tự cùng ảnh nằm ngoài task.

3.5. Dataset ảnh: EuroSAT

Nguồn: https://zenodo.org/records/7711810

Dataset	Nguồn gốc	Train / Val / Test	Input
EuroSAT	27,000	6400 / 1600 / 2000	16 × 16 × 3



Hình kế thừa 8 Mẫu thật của mười lớp; hình từ benchmark, không phải ảnh sinh.

EuroSAT RGB có 27.000 ảnh vệ tinh và 10 lớp sử dụng/phủ đất. Benchmark chọn 10.000 ảnh phân tầng theo nhãn rồi chia 6.400 train,1.600 validation,2.000 test. Các ảnh 64 × 64 RGB được resize 16 × 16 trước chuẩn hóa channel. Tiểu luận chỉ sử dụng ba kênh RGB, không nhận là dùng toàn bộ các dải phổ vệ tinh.

Các nhãn có thể tương đồng về texture và màu ở độ phân giải nhỏ. Random stratified split không kiểm soát riêng vị trí địa lý và khả năng các vùng gần nhau có liên quan. Vì vậy, metric trong bài là kết quả trong benchmark này, chưa chứng minh transfer sang vùng địa lý hoặc mùa quan sát khác.

3.6. CNN scratch: forward và cập nhật

```
patch = np.lib.stride_tricks.sliding_window_view(
    x, (3, 3), axis=(1, 2))
patch = patch.transpose(0, 1, 2, 4, 5, 3)
z = np.tensordot(patch, W, axes=([3,4,5], [0,1,2])) + b
a = np.maximum(z, 0)
h = a.reshape(B, 7, 2, 7, 2, 8).mean((2,4)).reshape(B, -1)
logits = h @ V + c
loss, dlogits = objective(logits, y, 'classification', weight)
grads = backward(parameters, cache, dlogits, 'cnn')
step(parameters, grads, lr=0.04, clip=1.0)
```

Shape patch là (B,14,14,3,3,C). Phép tensordot co ba trục kernel để thu output NHWC. Bộ cache giữ patch, pre-activation và feature head cho backward. Trong cài đặt thực tế B được lấy từ len(x), nên batch cuối nhỏ hơn 128 vẫn tính đúng trung bình loss.

Quá trình train chạy 20 epoch bằng SGD lr 0,04, cùng thứ tự mẫu cho ba implementation. Mỗi epoch đánh giá validation loss và giữ trọng số tốt nhất. Không augmentation hoặc dropout được thêm riêng cho framework. Tất cả đều train từ đầu từ cùng initialization.

Giảm ảnh xuống 16 × 16 và giữ một conv là lựa chọn phục vụ yêu cầu scratch đầy đủ trên CPU. Nó làm giới hạn kiến trúc dễ giải thích nhưng cũng giới hạn khả năng học. Báo cáo không che giấu đánh đổi này bằng cách dùng checkpoint của một mạng lớn hơn cho web hoặc lấy metric từ bài khác.

3.7. CNN Keras và PyTorch

```
# Keras, NHWC
h = Conv2D(8, 3, activation='relu')(inputs)
h = AveragePooling2D(2)(h)
h = Flatten()(h)
outputs = Dense(10)(h)

# PyTorch: conv nhận NCHW, flatten phải về NHWC.
a = F.conv2d(x.permute(0,3,1,2),
             W.permute(3,2,0,1), bias=b)
a = F.avg_pool2d(torch.relu(a), 2)
h = a.permute(0,2,3,1).reshape(len(x), -1)
logits = h @ V + c
```

Khác biệt layout là một nguồn lỗi phổ biến. Nếu PyTorch flatten trực tiếp NCHW nhưng gán head theo thứ tự NHWC của NumPy / Keras, các phần tử feature sẽ nối vào sai trọng số dù shape cuối đều là 392. Bài đổi lại layout trước flatten để bảo đảm cùng ánh xạ.

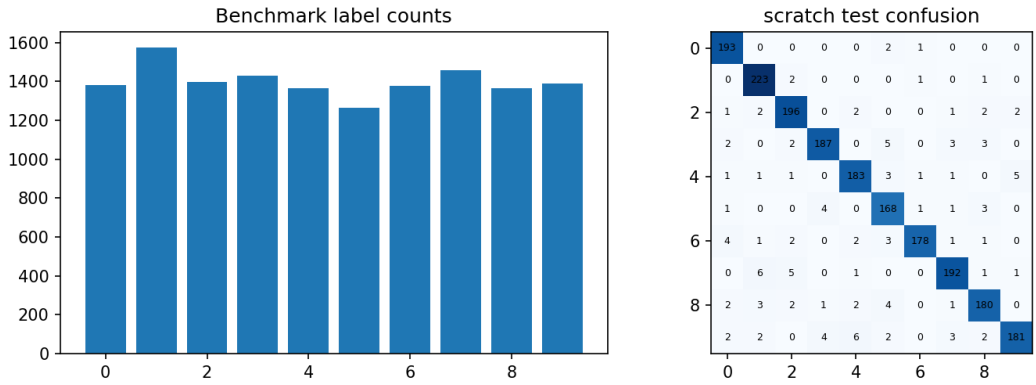
Các kernel được chuyển từ shape 3 × 3 ×C× 8 sang 8 ×C× 3 × 3 khi gọi convolution PyTorch. Đây là đổi cách lưu tensor, không phải đổi dữ liệu huấn luyện. Kiểm tra forward và gradient trước train giúp phát hiện nhanh nhầm thứ tự trục. Các checkpoint native được lưu sau khi gán lại trọng số epoch tốt nhất.

Cả ba implementation giữ softmax ở bước tính loss hoặc đánh giá; head không thêm softmax trước hàm cross-entropy nhận logits. Nguyên tắc này giúp tránh nhân đôi biến đổi hoặc gây mất ổn định số học. Bản web chỉ gọi softmax để hiển thị score của các lớp.

3.8. Đánh giá CNN trên MNIST

Cài đặt	Accuracy	Macro-F1
scratch	0.9405	0.9400
keras	0.9405	0.9400
pytorch	0.9405	0.9400

Chọn trên validation bằng Macro-F1: scratch. Baseline đơn giản trên test: Accuracy =0.1135, Macro-F1 =0.0204.



Hình kế thừa 9 Phân bố benchmark và confusion matrix của model chọn bằng validation.

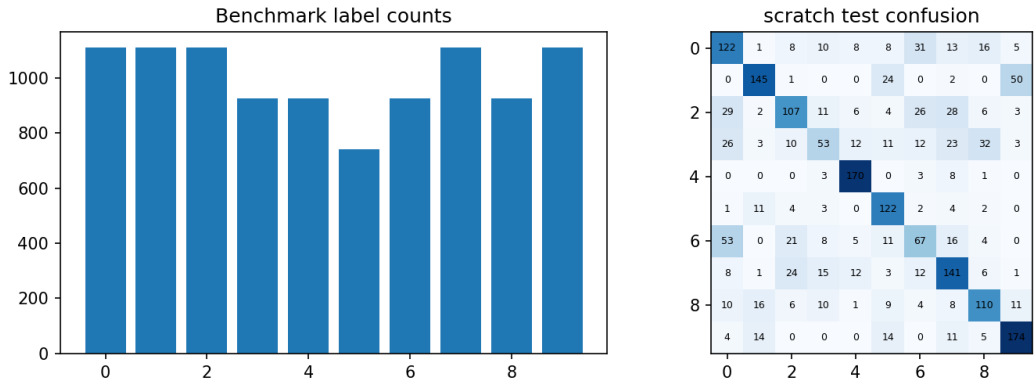
Macro-F1 lấy trung bình đều F1 các lớp, giúp tránh chỉ nhìn lớp có nhiều mẫu. Baseline dự đoán lớp phổ biến sử dụng thống kê train. Confusion matrix dùng số đếm; các ô hàng thật/cột dự đoán cho thấy hướng nhầm. Hình và bảng được tạo từ cùng prediction đã lưu, không dựng số liệu minh họa.

Các khác biệt rất nhỏ giữa framework có thể do làm tròn và thứ tự phép toán, đặc biệt khi score hai lớp gần nhau. Với chỉ một seed, không đủ cơ sở gọi khác biệt này là ưu thế ổn định. Cần tập trung vào mức chất lượng so với baseline và loại nhầm mà kiến trúc nhỏ còn gặp.

3.9. Đánh giá CNN trên EuroSAT

Cài đặt	Accuracy	Macro-F1
scratch	0.6055	0.5963
keras	0.6055	0.5963
pytorch	0.6055	0.5963

Chọn trên validation bằng Macro-F1: scratch. Baseline đơn giản trên test: Accuracy =0.1110, Macro-F1 =0.0200.

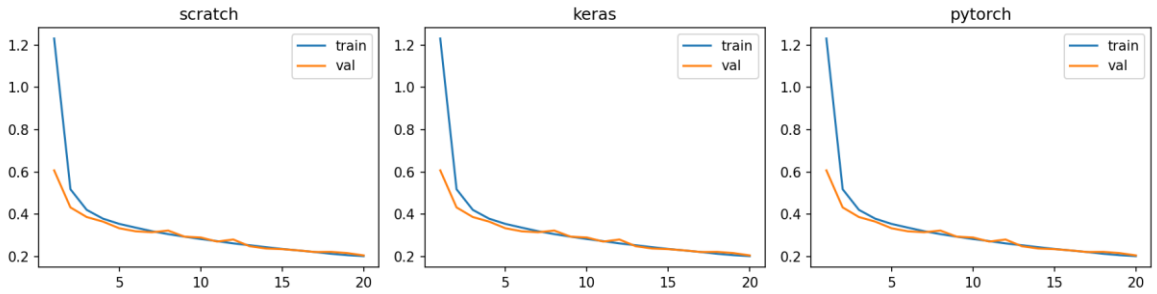


Hình kế thừa 10 Phân bố benchmark và confusion matrix của model chọn bằng validation.

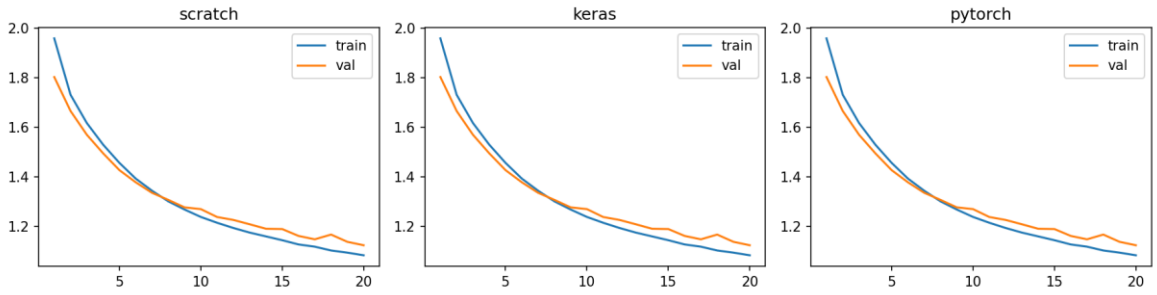
Macro-F1 lấy trung bình đều F1 các lớp, giúp tránh chỉ nhìn lớp có nhiều mẫu. Baseline dự đoán lớp phổ biến sử dụng thống kê train. Confusion matrix dùng số đếm; các ô hàng thật/cột dự đoán cho thấy hướng nhầm. Hình và bảng được tạo từ cùng prediction đã lưu, không dựng số liệu minh họa.

Các khác biệt rất nhỏ giữa framework có thể do làm tròn và thứ tự phép toán, đặc biệt khi score hai lớp gần nhau. Với chỉ một seed, không đủ cơ sở gọi khác biệt này là ưu thế ổn định. Cần tập trung vào mức chất lượng so với baseline và loại nhầm mà kiến trúc nhỏ còn gặp.

3.10. Đường học và chi phí CNN



Hình kế thừa 11 Learning curve của MNIST.



Hình kế thừa 12 Learning curve của EuroSAT.

Dataset / cài đặt	Tham số	Epoch tốt	Train giây
MNIST / scratch	4010	20	9.8716
MNIST / keras	4010	20	5.0716
MNIST / pytorch	4010	20	5.6658
EuroSAT / scratch	4154	20	10.3540
EuroSAT / keras	4154	20	4.0626
EuroSAT / pytorch	4154	20	4.6414

Thời gian bao gồm vòng train và validation trên CPU với các cấu hình hiện tại. NumPy và framework có overhead khác nhau; không suy ra tốc độ GPU hoặc tốc độ model lớn từ bảng này. Số tham số cho biết quy mô trọng số nhưng không phản ánh đầy đủ số phép tính.

3.11. Phân tích trường hợp sai

Chỉ số	Thực	Dự đoán	Sai số / score
12150	8.0000	0.0000	0.7883
12154	0.0000	5.0000	0.8620
12165	9.0000	1.0000	0.4726
12170	9.0000	5.0000	0.3359
12173	4.0000	9.0000	0.3602
12178	3.0000	8.0000	0.8629

Chỉ số	Thực	Dự đoán	Sai số / score
8001	3.0000	7.0000	0.4428
8004	0.0000	7.0000	0.5285
8007	3.0000	6.0000	0.3281
8008	2.0000	6.0000	0.2306
8010	3.0000	0.0000	0.6628
8011	3.0000	6.0000	0.3806

Bảng đầu liệt kê các mẫu MNIST sai; bảng sau là EuroSAT. Chỉ số mẫu tham chiếu cache benchmark; nhãn số ánh xạ theo danh sách classes trong summary.json. Score là softmax lớn nhất của dự đoán sai, không phải thước đo mức độ đúng. Một dự đoán sai có score cao cho thấy score chưa bảo đảm độ tin cậy.

MNIST có thể mất thông tin nét sau resize và gặp các chữ số có hình dạng gần nhau. EuroSAT đòi hỏi biểu diễn texture/màu và bối cảnh; ảnh 16 × 16 khó giữ chi tiết. Đây là các giả thuyết phù hợp với task, nhưng chưa được kiểm định bằng ablation độ phân giải trong bài hiện tại.

Các ví dụ lỗi được chọn sau khi cố định model. Không sửa kiến trúc để giảm lỗi trên chính những mẫu này rồi tiếp tục gọi chúng là test. Nếu muốn nghiên cứu sâu hơn, cần tách một bộ phân tích lỗi khỏi bộ đánh giá cuối hoặc dùng một test mới.

3.12. So sánh các kiến trúc khác nhau

Phần bổ sung theo hướng bài mẫu Phạm Văn Tư so sánh CNN cơ bản, VGG-style, MobileNet-style và ResNet-style. Đây là bốn kiến trúc khác nhau được train độc lập bằng PyTorch, tách khỏi phép kiểm tra ba thư viện ở các phần trước. Dữ liệu, split, chuẩn hóa và ảnh 16 × 16 được giữ nguyên; không lấy kết quả hoặc trọng số của bài mẫu.

Kiến trúc	Cấu trúc và khác biệt
CNN cơ bản	Conv 3 × 3, 8 kênh → ReLU → AvgPool → Dense
VGG-style	Bốn Conv 3 × 3, kênh 16/32, hai lần pooling, Dense 64
MobileNet-style	Mở rộng 1 × 1 → depthwise 3 × 3 → chiếu tuyến tính 1 × 1; một skip connection
ResNet-style	Stem 16 kênh, hai residual block 16/32 kênh, global average pooling

VGG dùng chuỗi kernel nhỏ để tăng độ sâu . MobileNetV 2 dùng inverted residual và linear bottleneck ; bản nhỏ ở đây dùng ReLU, không BatchNorm, nên chỉ gọi MobileNet-style. ResNet học nhánh F(x) cộng với đường tắt x ; bản hai block này không phải ResNet-18 đầy đủ. Cả bốn đều khởi tạo ngẫu nhiên, không dùng pre train ed.

```
# Residual block
residual = x
x = torch.relu(conv1(x))
x = conv2(x) + residual
x = torch.relu(x)
# Depthwise: mỗi kênh có kernel riêng
depth = nn.Conv2d(32, 32, 3, padding=1, groups=32)
```

Cùng seed 42, Adam learning rate 0,001, batch 128, 20 epoch và gradient clipping 1. Checkpoint lấy validation loss thấp nhất; kiến trúc mặc định chọn bằng validation Macro- F1 hoặc RMSE. Test chỉ dùng báo cáo sau khi cố định cấu hình. Không so trực tiếp mức tăng với nhóm SGD cũ để kết luận riêng tác dụng của kiến trúc, vì optimizer khác nhau. Toàn bộ code và learning curve nằm trong notebook 06.

3.13. Kết quả so sánh kiến trúc trên hai dataset

Dataset: MNIST

Mô hình	Accuracy	Macro-F1	Tham số

Mô hình	Accuracy	Macro-F1	Tham số
basic_cnn	0.9580	0.9578	4010
vgg_style	0.9775	0.9773	49850
mobilenet_style	0.8770	0.8754	4274
resnet_style	0.9570	0.9567	24170

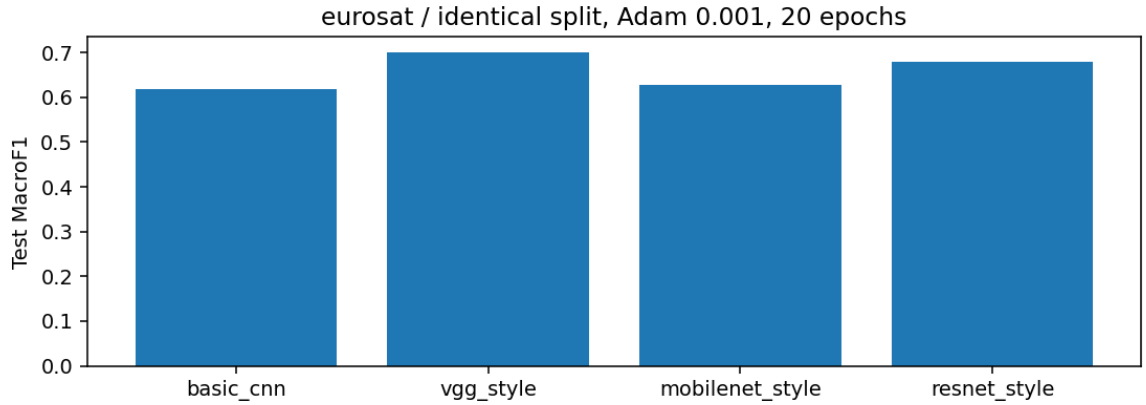
Chọn từ validation: vgg_style. Test Macro-F1 = 0.9773.

Dataset: EuroSAT

Mô hình	Accuracy	Macro-F1	Tham số
basic_cnn	0.6270	0.6195	4154
vgg_style	0.7065	0.7000	50138
mobilenet_style	0.6395	0.6273	4562
resnet_style	0.6915	0.6800	24458

Chọn từ validation: vgg_style. Test Macro-F1 = 0.7000.

Các kiến trúc cùng dùng một tập chia nhưng khác số tham số và thời gian huấn luyện; chi phí chi tiết được lưu trong CSV /notebook. Đây là so sánh cùng ngân sách epoch, không phải cùng FLOPs. Kết quả một seed chưa đủ kết luận ưu thế thống kê. Mô hình phức tạp hơn không mặc định tốt hơn trên mọi bộ dữ liệu.



Hình kế thừa 13 So sánh test trên dataset thứ hai; mô hình được chọn bằng validation.

3.14. Tiểu kết CNN và hướng mở rộng

Chương 3 đã triển khai trọn vẹn một CNN nhỏ bằng NumPy scratch, Keras và PyTorch, huấn luyện trên hai tập ảnh có khác biệt về cấu trúc. Bản scratch có backward qua head, average pooling, ReLU và convolution kernel. Sai phân hữu hạn và gradient autograd xác nhận các phép biến đổi này trên mini- batch kiểm tra.

So sánh cùng kiến trúc giúp tách câu hỏi " code có nhất quán không" khỏi câu hỏi "kiến trúc có đủ tốt không". Khi giữ mọi điều kiện tương ứng, ba đường học và metric thường rất gần. Chất lượng thực tế còn chịu ảnh hưởng của độ phân giải, lượng dữ liệu, cấu trúc mạng và đặc trưng của nhãn.

Phần bổ sung đã thử các kiến trúc convolution sâu hơn, depthwise và residual trong protocol Adam riêng. Augmentation, batch normalization và transfer learning vẫn chưa được thử. Nếu thêm chỉ cho một implementation thì không còn là phép đối chiếu framework hiện tại. Cần ghi lại cả chi phí và metric, thay vì tăng kiến trúc rồi chỉ báo cáo một chỉ số tốt nhất.

Web phục vụ đúng model 16 × 16 của bài, có hiển thị mẫu input để người đọc thấy phạm vi. Ảnh người dùng khác domain, bị xoay mạnh hoặc chứa nhiều đối tượng không được bảo đảm phù hợp. Mô hình EuroSAT không phân đoạn vị trí trên ảnh; nó chỉ gán một trong mười lớp cho toàn ảnh đầu vào.

Hạng mục	Kết luận phạm vi
Scratch	Đã có forward, backward và SGD thật
Benchmark	Tập con cố định, ảnh 16 × 16
Framework	Cùng initialization và kiến trúc
Hạn chế	Một seed; chưa spatial split /augmentation/transfer

CHƯƠNG 4. RNN

4.1. Chuỗi và trạng thái

Dữ liệu tuần tự gồm các quan sát có thứ tự. Một cửa sổ X cóT bước, mỗi bướcD đặc trưng; batch có shape(B,T,D). RNN cập nhật trạng thái h_t từ input x_t và trạng thái trước. Cùng trọng số được dùng ở mọi bước, tạo một hàm có cấu trúc phù hợp khi muốn tích lũy thông tin quá khứ.

Trong mô hình many-to-one của bài, chỉ trạng thái cuối được đưa vào head để tạo một target. Với AAPL, target là log-return phiên kế tiếp; với khách hàng, target là có mua trong tuần ngay sau cửa sổ. Mỗi cửa sổ khởi tạo h 0 bằng 0, không mang trạng thái của khách hoặc cửa sổ khác qua batch.

RNN không tự bảo đảm hiểu quan hệ thời gian. Nếu input có ngày đích, hoặc scaler fit cả tương lai, model có thể nhận thông tin không tồn tại ở lúc dự đoán. Vì vậy, việc xác định thời điểm nhãn có thể biết và các bước lịch sử được phép dùng quan trọng hơn chỉ lựa chọn một recurrent layer.

Tiểu luận chọn Vanilla RNN để bản scratch BPTT có thể đọc đầy đủ và đối chiếu trực tiếp. Phần bổ sung so sánh Simple RNN, LSTM, GRU và BiLSTM bằng PyTorch; tám checkpoint này được báo cáo riêng ở mục 4.12–4.13. Các kết quả RNN A 06 trước đó không được thay vào bảng hiện tại vì optimizer và protocol đối chiếu đã thay đổi.

Dataset	Nguồn gốc	Train / Val / Test	Input
AAPL	2,766	1880 / 403 / 404	30 × 8
Online Retail II	1,067,371	25937 / 5932 / 7484	8 × 6

4.2. Vanilla RNN và hợp hàm theo thời gian

```
h = np.zeros((batch_size, 16))
states = [h]
for t in range(sequence_length):
    h = np.tanh(x[:, t] @ W + h @ U + b)
    states.append(h)
logits = h @ V + c
```

Tại bước t, h_t=tanh(x_tW+h_{t-1}U+b). Nếu gọi f_t là hàm cập nhật với input x_t đã cố định, trạng thái cuối là hợp f_T∘...∘f_1 tác động lênh 0. Tham sốW,U,b được chia sẻ, còn các giá trịx_t khác nhau. Cách biểu diễn này liên hệ trực tiếp với yêu cầu giải thích hàm và hợp hàm của học phần.

Ma trậnW có shapeD× 16, U có 16 × 16, b có 16 phần tử. HeadV có 16 ×C và biasc cóC phần tử. Với regression C=1; classification hai lớpC=2. Số tham số không tăng theoT, dù số phép tính và bộ nhớ lưu trạng thái trong backward tăng theo chiều dài chuỗi.

Trong bài, PyTorch hiện thực recurrence bằng phép tensor với ParameterDict thay vì nn. RNN mặc định có hai bias. Keras Simple RNN và scratch đều dùng một bias. Lựa chọn này làm số tham số và quy tắc tối ưu trùng nhau, tránh một nguồn khác biệt đã gặp khi chuyển giữa thư viện.

Dataset	D	T	Output	Tham số
AAPL	8	30	1	417
Khách hàng	6	8	2	402

4.3. BPTT và kiểm soát gradient

```
dh = dlogits @ V.T
dW = np.zeros_like(W); dU = np.zeros_like(U)
db = np.zeros_like(b)
for t in range(T-1, -1, -1):
    dz = dh * (1 - states[t+1]**2)
    dW += x[:, t].T @ dz
    dU += states[t].T @ dz
    db += dz.sum(axis=0)
    dh = dz @ U.T
```

Backward đi từ bước cuối về đầu. Đạo hàm tanh bằng $1-h^2$; gradientW vàU nhận tổng đóng góp từ mọi thời điểm vì cùng trọng số được dùng lặp lại. Gradient đi tới trạng thái trước bằng phép nhânU chuyển vị. Các chỉ số states[t] và states[t+1] phải khớp h trước/sau cập nhật; sai một chỉ số có thể vẫn cho shape hợp lệ nhưng đạo hàm sai.

Tích nhiều Jacobian có thể co nhỏ hoặc tăng mạnh. Gradient clipping global norm 1 hạn chế bùng nổ bằng cách scale đồng thời toàn bộ gradient; nó không tự giải quyết mọi phụ thuộc dài hạn. Độ dài cửa sổ, initialization và độ bão hòa tanh đều ảnh hưởng việc truyền tín hiệu.

Scratch trong bài không gọi autograd. Hai framework tự động đạo hàm cùng recurrence. Check gradient dùng chuỗi ngắn và float 64 cho finite differences, còn train dùng float 32. Sai số nhỏ trong kiểm tra cho thấy việc cài đạo hàm nhất quán trên ví dụ kiểm tra; không đồng nghĩa đã chứng minh đúng cho mọi trường hợp số học hoặc dữ liệu bất thường.

4.4. LSTM, GRU và phạm vi mô hình

LSTM bổ sung cell state và các cổng input, forget, output. Cell được cập nhật bằng đường cộng của thông tin được giữ và thông tin mới, giúp tạo một đường truyền khác với recurrence tanh đơn giản. Các gate có giá trị liên tục, không phải nhãn 0/1 cứng. Cơ chế này nhằm cải thiện học phụ thuộc dài nhưng không loại bỏ mọi vấn đề dữ liệu và tối ưu .

GRU kết hợp reset và update gate, không có cell state riêng. Các thư viện có biến thể reset-before hoặc reset-after; việc chuyển trọng số cần thống nhất công thức và thứ tự gate. Bi RNN /BiLSTM đọc hai chiều của một cửa sổ; nếu cửa sổ hoàn toàn là quá khứ thì đọc ngược không tự tạo leakage, nhưng đưa thời điểm đích vào cửa sổ vẫn là sai.

Attention cho phép truy cập các vị trí khác qua trọng số phụ thuộc nội dung thay vì chỉ qua trạng thái cuối. Tuy nhiên, nghiên cứu thêm kiến trúc này cần một hợp đồng so sánh khác về số tham số, compute và dữ liệu. Tiểu luận đã train LSTM, GRU và BiLSTM trong nhóm so sánh kiến trúc ở mục 4.12–4.13; attention và Transformer vẫn chỉ là bối cảnh lý thuyết.

Một mô hình đơn giản nhưng có protocol rõ tạo điểm khởi đầu có thể kiểm tra. Khi baseline quá mạnh hoặc predictor ít thông tin, tăng độ phức tạp chưa chắc đem lại cải thiện. Các thí nghiệm sau cần chứng minh lợi ích trên dữ liệu ngoài mẫu thay vì dựa vào trực giác rằng nhiều gate sẽ luôn tốt hơn.

Kiến trúc	Trạng thái	Vai trò trong tiểu luận
Vanilla RNN	Hidden tanh	Train bằng cả ba cách
LSTM / BiLSTM	Hidden + cell + gates	Đã train bằng PyTorch
GRU	Hidden + reset/update	Đã train bằng PyTorch
Attention	Kết hợp vị trí có trọng số	Bối cảnh lịch sử

4.5. Dataset chuỗi: AAPL

Nguồn: https://github.com/tuananhtrieu1305/ISD_Assignment01/tree/main/A06/datasets

Dataset	Nguồn gốc	Train / Val / Test	Input
AAPL	2,766	1880 / 403 / 404	30 × 8

Mẫu	Input chuẩn hóa (tối đa 8 số đầu)	Target gốc
2283	-0.6450, -0.4360, -0.6810, -0.4830, -0.6700, -1.0540, -0.3820, -2.0280	-0.0076
2284	2.2240, 0.2520, 1.7330, 2.7670, 0.2690, -0.4770, -0.0280, -0.6080	-0.0213
2285	0.4160, -0.3820, 0.2640, 0.8770, 0.4460, -0.3490, -0.0350, -0.3490	0.0165

AAPL có 2.766 dòng từ 2015-01-02 đến 2025-12-31. Tám đặc trưng gồm log-return, OpenGap, Range, Body, độ lệch MA20, độ lệch MA50, volatility 20 và log-volume. Các phép tính chỉ dùng quan sát ở hoặc trước ngày đặc trưng. Window 30 phiên kết thúc ngay trước target; bỏ phần đầu chưa đủ lịch sử.

Target là log(Close_t/Close_{t-1}), chuẩn hóa bằng mean/std train. Giá dự báo được khôi phục từ Close gần nhất nhân exp(return). Cách thiết kế giúp head học biến động một bước thay vì mức giá có xu hướng dài hạn, nhưng không bảo đảm vượt persistence.

Các target date được chia tăng dần 70/15/15. Dự báo test là rolling one-step với lịch sử thật đã quan sát. Không dùng random split và không diễn giải đồ thị này thành dự báo nhiều tháng từ một cửa sổ ban đầu.

4.6. Dataset chuỗi: Online Retail II

Nguồn: https://github.com/tuananhtrieu1305/ISD_Assignment01/tree/main/A06/datasets

Dataset	Nguồn gốc	Train / Val / Test	Input
Online Retail II	1,067,371	25937 / 5932 / 7484	8 × 6

Mẫu	Input chuẩn hóa (tối đa 8 số đầu)	Target gốc
31869	-0.4570, -0.4590, -0.4660, -0.4450, -0.4680, -0.4670, -0.4570, -0.4590	0.0000
31870	1.8250, 1.8210, 2.2610, 1.7610, 2.0100, 2.3140, -0.4570, -0.4590	0.0000
31871	-0.4570, -0.4590, -0.4660, -0.4450, -0.4680, -0.4670, -0.4570, -0.4590	0.0000

Online Retail II có 1.067.371 dòng trong hai sheet. Loại bản ghi trùng, thiếu CustomerID, đơn hủy, Quantity hoặc Price không dương. Gom sáu biến mỗi khách/tuần: Orders, Quantity, Revenue, DistinctItems, ActiveDays, AverageOrderValue. Giá trị tiền dùng GBP; tuần không mua được điền 0.

Chỉ tạo mẫu cho khách có đơn trong tám tuần trước, anchor cách nhau bốn tuần nhưng target vẫn là tuần kế tiếp. Tuần cuối chưa đủ ngày không dùng làm nhãn; tuần đầu nguồn chỉ quan sát từ 01/12/2009, được giữ trong phần lịch sử khởi tạo. Chia theo target week, cùng khách có thể xuất hiện qua các thời kỳ khác nhau.

Input log 1 p và chuẩn hóa train -only. Nhãn là mua tuần tới, không phải churn vĩnh viễn. Không dùng mã khách làm predictor. Cách lấy mẫu tập trung vào nhóm có hoạt động gần đây; chưa đánh giá những khách đã im lặng lâu hơn tám tuần.

4.7. RNN Keras và PyTorch

```
# Keras: SimpleRNN cùng một bias như NumPy.
inputs = tf.keras.layers.Input(shape=(T, D))
h = tf.keras.layers.SimpleRNN(16, activation='tanh')(inputs)
outputs = tf.keras.layers.Dense(C)(h)

# PyTorch: recurrence dùng phép tensor và autograd.
h = torch.zeros((len(x), 16), dtype=x.dtype)
for t in range(x.shape[1]):
```

```
    h = torch.tanh(x[:, t] @ p['w'] + h @ p['u'] + p['b'])
logits = h @ p['v'] + p['c']
```

Output AAPL là một giá trị liên tục; khách hàng có hai logits dùng weighted softmax cross-entropy. Hai framework không thêm activation ở head. Class weights chỉ tính từ train; dự đoán nhãn dùng argmax nên threshold cố định 0,5, khác với A 06 từng chọn threshold trên validation.

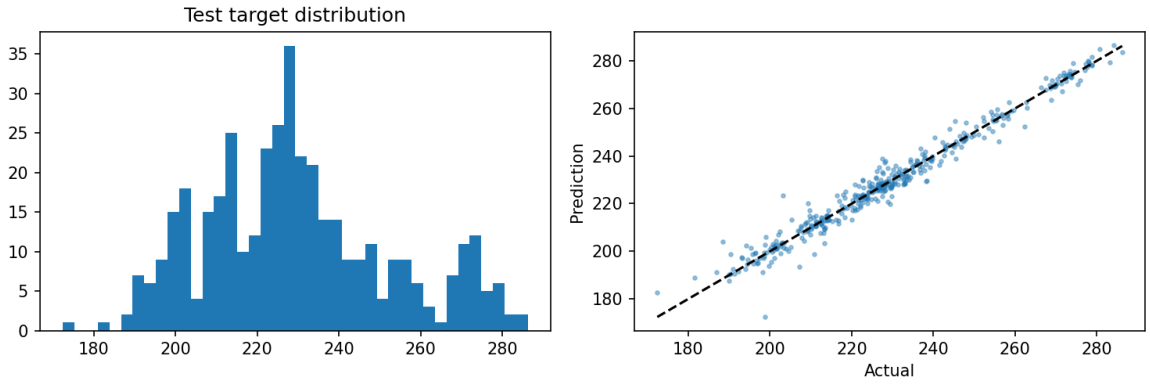
Cả ba dùng 18 epoch, batch 128, SGD lr 0,02 và clipnorm 1. Mỗi epoch dùng cùng phép hoán vị chỉ số train với seed 42+ epoch; thứ tự bên trong cửa sổ luôn giữ nguyên. Validation loss được tính bằng cùng hàm NumPy trên trọng số export để giảm khác biệt về reduction khi đối chiếu.

Các native checkpoint được lưu ở epoch tốt nhất. Khi phục vụ, trọng số được đọc từ npz và áp dụng cùng forward. Nạp lại native model rồi so output là bước cần thiết: chỉ lưu được một file không đủ xác nhận rằng file đó tương ứng đúng model được báo cáo.

4.8. Kết quả RNN: AAPL

Cài đặt	RMSE	MAE	R^2
scratch	3.9322	2.6517	0.9706
keras	3.9322	2.6517	0.9706
pytorch	3.9322	2.6517	0.9706

Chọn trên validation bằng RMSE: scratch. Baseline đơn giản trên test: RMSE=3.9046, MAE=2.6416, R^2=0.9711.



Hình kế thừa 14 Phân bố và đánh giá trên test của AAPL.

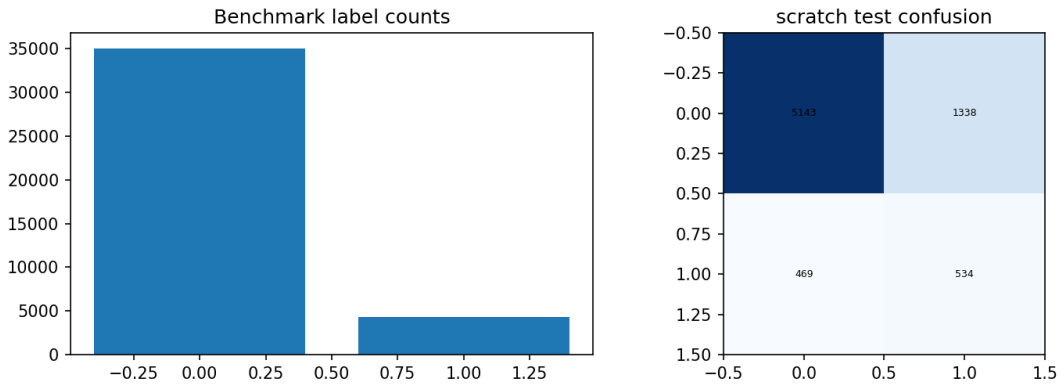
Baseline persistence dự đoán giá phiên kế tiếp bằng giá phiên gần nhất. Đây là đối chiếu quan trọng vì giá liên tiếp thường gần nhau: R^2 cao trên mức giá có thể vẫn không vượt một dự đoán không học. Kết luận phải dựa trên RMSE/MAE so baseline, không chỉ nhìn các điểm nằm gần đường chéo.

Không đánh giá chiến lược giao dịch, phí hoặc lợi nhuận. Các sai số trongUSD phản ánh mục tiêu regression một bước. Nếu muốn kết luận khả năng dự báo hướng hoặc ứng dụng tài chính, cần thêm một protocol độc lập và những metric phù hợp.

4.9. Kết quả RNN: Online Retail II

Cài đặt	Accuracy	Macro-F1	F1	AP
scratch	0.7586	0.6110	0.3715	0.3165
keras	0.7586	0.6110	0.3715	0.3165
pytorch	0.7586	0.6110	0.3715	0.3165

Chọn trên validation bằng Macro-F1: scratch. Baseline đơn giản trên test: Accuracy =0.8660, Macro-F1 =0.4641.

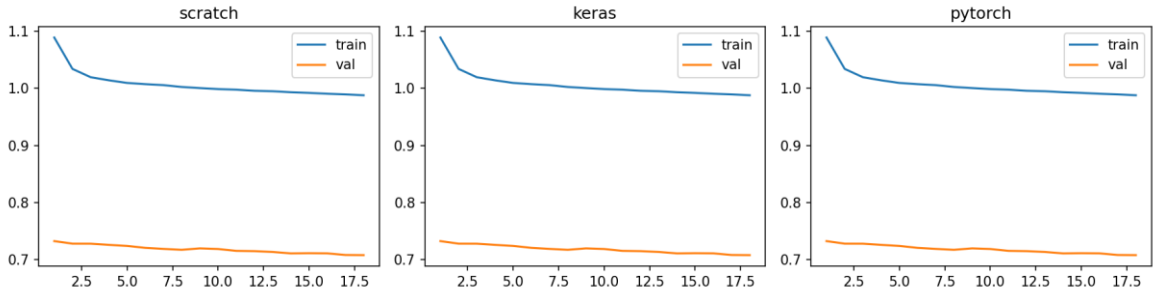


Hình kế thừa 15 Phân bố và đánh giá trên test của Online Retail II.

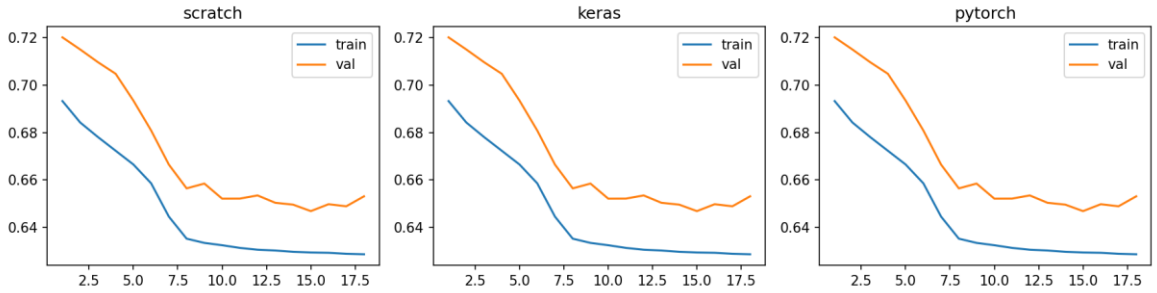
Tỷ lệ không mua lớn nên baseline lớp phổ biến có thể có Accuracy cao nhưng F1 lớp dương thấp. Weighted loss thay đổi ưu tiên khi học; không làm score trở thành xác suất đã calibration. AP vàROC-AUC bổ sung góc nhìn xếp hạng; Macro-F1 là tiêu chí chọn implementation trên validation trongtiểu luận.

Kết quả không chứng minh mô hình phát hiện người rời bỏ dài hạn. Khách không mua ở một tuần có thể mua ở tuần sau. Dođó giao diện và báo cáo đều dùng thuật ngữ “mua tuần kế tiếp” theo đúng định nghĩa nhãn.

4.10. Learning curve và chi phí RNN



Hình kế thừa 16 Loss của AAPL theo epoch.



Hình kế thừa 17 Loss của Online Retail II theo epoch.

Dataset / cài đặt	Tham số	Epoch tốt	Train giây
AAPL / scratch	417	18	0.9002
AAPL / keras	417	18	1.9738
AAPL / pytorch	417	18	1.6210
Online Retail II / scratch	402	15	3.0378
Online Retail II / keras	402	15	6.7670
Online Retail II / pytorch	402	15	9.1446

Số tham số bằng nhau giữa ba cách cài đặt do cùng một bias recurrent và cùng head. Thời gian là số đo mô tả trên CPU, chưa có warm-up hoặc lặp benchmark nhiều lần. Không suy ra mọi RNN scratch đều nhanh/chậm hơn framework từ cấu hình nhỏ này.

4.11. Phân tích sai số và dịch chuyển thời gian

Chỉ số	Thực	Dự đoán	Sai số / score
2503	198.8500	172.6090	26.2410

Chỉ số	Thực	Dự đoán	Sai số / score
2499	203.1900	223.5614	20.3714
2500	188.3800	203.9194	15.5394
2296	207.1500	193.5691	13.5809
2525	210.7900	198.9008	11.8892
2481	227.4800	239.0199	11.5399

Chỉ số	Thực	Dự đoán	Sai số / score
31887	0.0000	1.0000	0.8201
31888	0.0000	1.0000	0.5607
31893	0.0000	1.0000	0.7380
31898	1.0000	0.0000	0.5400
31906	0.0000	1.0000	0.6278
31911	0.0000	1.0000	0.5870

Bảng AAPL ưu tiên các sai số giá lớn nhất; bảngkhách hàng liệt kê ví dụ dự đoán sai đầu tiên trong test của implementation đã chọn. Chỉ số tham chiếu cache được lưu để có thể mở lại input. Không dùng những hàng này để đổi checkpoint.

Thị trường và hành vi khách hàng có thể thay đổi theo thời gian. Mean/std train cố định không xóa được distribution shift, nhưng fit lại bằng toàn test sẽ tạo một đánh giá không phù hợp với thời điểm dự đoán. Trong triển khai thực tế cần theo dõi thống kê đầu vào và chất lượng khi có nhãn mới; tiểu luận chưa xây dựng vòng tự tái huấn luyện.

Input khách hàng gồm các tổng tuần, không chứa các chiến dịch, mùa khuyến mại hoặc thông tin cá nhân bổ sung. AAPL chỉ có OHLCV, không có tin tức. Vì vậy không quy lỗi một mẫu cụ thể cho một sự kiện không quan sát được trong dataset. Những suy luận nguyên nhân cần nguồn dữ liệu khác và phương pháp kiểm định phù hợp.

4.12. So sánh các kiến trúc khác nhau

Bốn kiến trúc được so sánh là Simple RNN, LSTM, GRU và BiLSTM. Dữ liệu vẫn là AAPL và Online Retail II, khác với cổ phiếu Việt Nam và bộ khách hàng mô phỏng trong bài Tư.

Mỗi mô hình có một tầng hồi tiếp, hidden size 32; BiLSTM có 32 trạng thái mỗi chiều nên nhiều tham số hơn. Head trả log-return hoặc hai logits mua hàng.

Kiến trúc	Cơ chế
Simple RNN	h mới = tanh(Wx + Uh + b)
LSTM	Ba cổng input /forget/ output, trạng thái cell và hidden
GRU	Cổng reset/update, một trạng thái hidden
BiLSTM	Hai LSTM đọc xuôi/ngược trong cửa sổ lịch sử; nối hai hidden cuối

BiLSTM chỉ đọc hai chiều bên trong 30 phiên hoặc 8 tuần đã quan sát trước thời điểm dự báo. Nó không được đọc phiên/tuần chứa nhãn. Vì vậy hai chiều không tự tạo leakage trong bài many-to-one này. Dùng h_n của cả hai chiều, không lấy output tại bước cuối một cách thiếu chiều ngược.

```
layer = nn.LSTM(input_size=D, hidden_size=32,
                batch_first=True, bidirectional=True)
_, (h, cell) = layer(x)
features = torch.cat((h[-2], h[-1]), dim=1)
logits = head(features)
```

Cùng seed 42, Adam learning rate 0,001, batch 128, 20 epoch và gradient clipping 1. Checkpoint lấy validation loss thấp nhất; kiến trúc mặc định chọn bằng validation Macro- F1 hoặc RMSE. Test chỉ dùng báo cáo sau khi cố định cấu hình. Không so trực tiếp mức tăng với nhóm SGD cũ để kết luận riêng tác dụng của kiến trúc, vì optimizer khác nhau. Toàn bộ code và learning curve nằm trong notebook 06.

4.13. Kết quả so sánh kiến trúc trên hai dataset

Dataset: AAPL (USD)

Mô hình	RMSE	MAE	R^2	Tham số
simple_rnn	3.9641	2.6790	0.9702	1377
lstm	3.8946	2.6562	0.9712	5409
gru	3.9381	2.6764	0.9706	4065
bilstm	3.9250	2.6446	0.9708	10817

Chọn từ validation: lstm. Test RMSE = 3.8946.

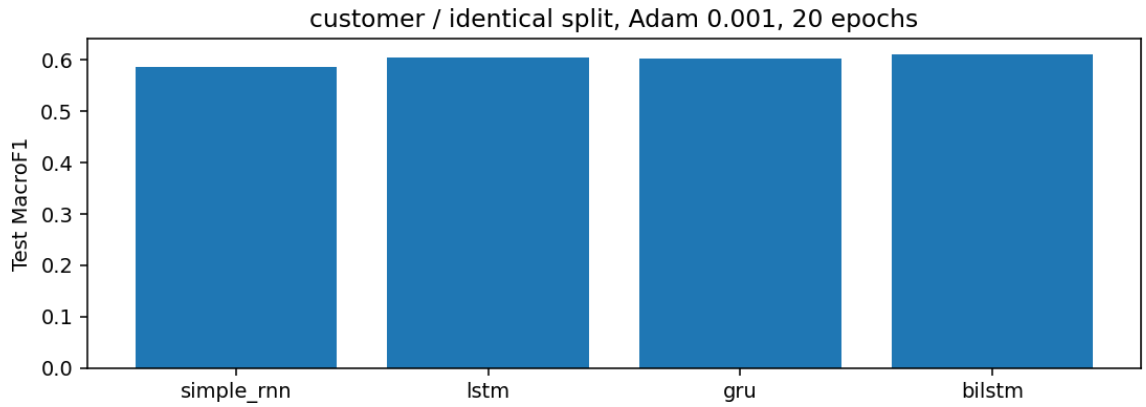
Baseline giữ giá phiên trước có RMSE 3.9046 USD. Mô hình chọn trên validation có RMSE thấp hơn baseline; R^2 cao không thay thế đối chiếu này.

Dataset: Online Retail II

Mô hình	Accuracy	Macro-F1	Tham số
simple_rnn	0.7229	0.5863	1346
lstm	0.7493	0.6052	5186
gru	0.7465	0.6029	3906
bilstm	0.7569	0.6107	10370

Chọn từ validation: bilstm. Test Macro-F1 = 0.6107.

Các kiến trúc cùng dùng một tập chia nhưng khác số tham số và thời gian huấn luyện; chi phí chi tiết được lưu trong CSV /notebook. Đây là so sánh cùng ngân sách epoch, không phải cùng FLOPs. Kết quả một seed chưa đủ kết luận ưu thế thống kê. Mô hình phức tạp hơn không mặc định tốt hơn trên mọi bộ dữ liệu.



Hình kế thừa 18 So sánh test trên dataset thứ hai; mô hình được chọn bằng validation.

4.14. Tiểu kết RNN

Chương 4 đã xây dựng một Vanilla RNN many-to-one và triển khai BPTT bằng NumPy. Hai framework thực hiện cùng recurrence và cùng quy tắc tối ưu. Hai dataset bao phủ regression giá phiên kế tiếp và classification mua tuần kế tiếp, với target được định nghĩa theo thời điểm rõ ràng.

Điểm quan trọng của thực nghiệm là giữ đúngquan hệ quá khứ-tương lai và ghi rõ dạng rolling one-step. Các quan sát trước nhãn được sử dụng, còn nhãn tương lai không được đưa vào scaler hoặc window. Việc cùng khách xuất hiện trong nhiều giai đoạn là đặc điểm thiết kế của bài mua lại, không phải phép thử khách hoàn toàn mới.

Baseline và phân tích lỗi giúp tránh diễn giải quá mức metric. Ba framework tạo kết quả gần nhau chứng tỏ tính nhất quán của phép triển khai dưới điều kiện hiện tại; nó không giải quyết giới hạn thông tin trong lịch sử. Một mạng có gate hoặc một bộ dữ liệu lớn hơn chỉ có ý nghĩa nếu được kiểm tra bằng protocol phù hợp.

Hướng phát triển tiếp theo gồm nhiều seed, walk- forward validation, phân tích drift, calibration score và đối chiếu với mô hình chuỗi đơn giản. Tất cả là công việc chưa thực hiện, được tách rõ khỏi các kết quả đã train trong tiểu luận. Checkpoint hiện tại được sử dụng nguyên vẹn trong web để người dùng thử cùng phạm vi input.

Quy tắc	**Cách áp dụng**
Không nhìn tương lai	Window kết thúc trước target
Chọn model	Validation Macro-F1 hoặc RMSE
Phục vụ	Cùng mean/std và trọng số export
Phạm vi	Một bước; một seed; không online learning

CHƯƠNG 5. KNOWLEDGE BASE, KNOWLEDGE GRAPH VÀ RETRIEVAL-AUGMENTED GENERATION

Chương này mở rộng quyển tiểu luận từ mô hình dự đoán sang hệ thống trả lời có bằng chứng. Mô hình ML, CNN hoặc RNN ánh xạ đầu vào thành nhãn hay giá trị; RAG nhận một câu hỏi, truy hồi các đoạn liên quan rồi tạo câu trả lời kèm nguồn. Knowledge Graph bổ sung các thực thể và quan hệ để truy vấn không chỉ phụ thuộc vào từ khóa.

Thiết kế được biểu diễn như hợp hàm: answer = compose(review, generate, retrieve, normalize)(question). Mỗi hàm có đầu vào, đầu ra và trách nhiệm riêng. Nhờ đó có thể thay BM25 bằng LSA, thêm mở rộng đồ thị hoặc đổi cách tổng hợp mà không làm thay đổi toàn bộ hệ thống.

Thành phần	Đầu vào	Đầu ra	Mục đích
Normalize	Câu hỏi Unicode	Token chuẩn hóa	Giảm khác biệt chữ hoa và dấu
Retriever	Token và chỉ mục	Danh sách tài liệu	Tìm bằng chứng phù hợp
Knowledge Graph	Thực thể hạt giống	Đường đi và evidence	Mở rộng quan hệ nhiều bước
Generator	Câu hỏi và bằng chứng	Câu trả lời có trích dẫn	Không trả lời ngoài dữ liệu

5.1. Mở đầu

Knowledge Base (KB) là kho tri thức có cấu trúc dùng để lưu tài liệu, nguồn, ngày cập nhật và các phát biểu có thể truy vết. Knowledge Graph (KG) là một cách tổ chức KB dưới dạng mạng thực thể–quan hệ. Retrieval-Augmented Generation (RAG) dùng truy hồi để đưa bằng chứng từ KB/KG vào quá trình trả lời. Ba lớp này bổ sung cho mô hình dự đoán ở Chương 2–4: hệ thống vừa đưa ra kết quả, vừa chỉ ra dữ liệu nào hỗ trợ kết quả đó.

Mục tiêu của chương là xây dựng pipeline có thể chạy lại trên hai miền đã dùng trong quyển: bệnh đái tháo đường và chuỗi AAPL. Bài so sánh năm cấu hình B0–B4, kiểm tra citation, khả năng từ chối câu ngoài phạm vi, phân tích lỗi và cung cấp API triển khai. Phạm vi là minh họa học thuật; câu trả lời sức khỏe không thay thế chẩn đoán và nội dung tài chính không phải khuyến nghị đầu tư.

5.2. Knowledge Base và Knowledge Graph

Knowledge Base của bài gồm tài liệu, metadata nguồn và các bộ ba quan hệ. Mỗi tài liệu có id, domain, title, url, as_of và text. Knowledge Graph biểu diễn tri thức bằng bộ ba (subject, predicate, object). Subject và object là thực thể; predicate mô tả quan hệ có hướng. Mỗi cạnh

còn mang trường evidence trỏ về một tài liệu trong corpus, nhờ đó quan hệ dùng để trả lời luôn có thể truy vết.

Đồ thị được lưu dạng property graph trong JSON. Khi nạp dữ liệu, mã tạo danh sách kề hai chiều để phục vụ tìm kiếm. Với câu hỏi có thực thể khớp alias, BFS duyệt tối đa hai hop. Điểm một cạnh tại độ sâu d là 1/(d+1), vì quan hệ xa hơn thường ít trực tiếp hơn.

Thống kê	Giá trị	Ý nghĩa
Tài liệu	12	6 sức khỏe và 6 tài chính
Bộ ba	23	Quan hệ đều có evidence hợp lệ
Miền	health, finance	Khớp hai CSV được nộp
Số hop	1–2	Giới hạn nhiễu khi mở rộng

```
queue = deque((seed, 0) for seed in seeds)
while queue:
    node, depth = queue.popleft()
    if depth >= hops:
        continue
    for next_node, triple in adjacency[node]:
        evidence[triple["evidence"]] += 1 / (depth + 1)
        queue.append((next_node, depth + 1))
```

5.3. Retrieval-Augmented Generation

BM25 là phương pháp truy hồi từ vựng. Điểm tăng khi từ của câu hỏi xuất hiện nhiều trong tài liệu, nhưng được điều chỉnh theo độ dài tài liệu và độ hiếm IDF. Bài dùng k 1=1,5 và b=0,75. Ưu điểm là nhanh, dễ giải thích và phù hợp corpus nhỏ; nhược điểm là khó nối các cách diễn đạt khác nhau.

LSA xây ma trận TF-IDF tài liệu–từ, chuẩn hóa từng hàng và phân rã SVD. Câu hỏi được chiếu vào không gian tám chiều rồi xếp hạng theo cosine similarity. Đây là mô hình dense cổ điển chạy hoàn toàn offline bằng NumPy. Nó không được mô tả như embedding ngôn ngữ hiện đại và không dùng dịch vụ bên ngoài.

```
tfidf = normalize_rows(term_frequency * inverse_document_frequency)
_, _, vt = np.linalg.svd(tfidf, full_matrices=False)
projection = vt[:8].T
dense_docs = tfidf @ projection
score = cosine(dense_docs, query_tfidf @ projection)
```

Hai danh sách BM25 và LSA được kết hợp bằng Reciprocal Rank Fusion: RRF(d)=Σ w/(60+rank). Phương pháp dùng thứ hạng thay cho thang điểm gốc nên tránh phải ép điểm BM25, cosine và đồ thị về cùng đơn vị.

5.3.1. GraphRAG và các baseline B0–B4

GraphRAG trong bài là pipeline xác định, không gọi mô hình ngôn ngữ lớn. Hệ thống tính ba ranking BM25, LSA và KG; hợp nhất bằng RRF với trọng số lần lượt 1,0; 0,7; 1,2; lấy top-3; rồi ghép các đoạn bằng chứng và danh sách URL. Cách làm này đủ để đánh giá truy hồi, dễ chạy trên máy cá nhân và tránh phát sinh câu không có trong nguồn.

Một truy vấn trực tiếp có thể được BM25 giải quyết. Truy vấn nhiều bước như mối quan hệ giữa AAPL, OHLCV, RNN và giới hạn đầu tư nhận thêm đường đi từ graph. Nếu không có tài liệu, answer trả thông báo thiếu bằng chứng. Trong miền sức khỏe và tài chính, câu trả lời luôn kèm lưu ý về phạm vi học thuật.

```
ranking = rrf([
    (bm25(query), 1.0),
    (lsa(query), 0.7),
    (graph(query, hops=2), 1.2),
])
documents = ranking[:3]
answer = compose_notice(compose_citations(documents))
```

Phương pháp	Tín hiệu	Điểm mạnh	Giới hạn
BM25	Từ vựng	Nhanh, giải thích được	Nhạy cách dùng từ
LSA	Không gian SVD	Bắt quan hệ tiềm ẩn đơn giản	Corpus nhỏ làm vector kém ổn định
KG	Thực thể và cạnh	Có đường đi	Phụ thuộc alias và độ phủ graph
GraphRAG	Ba ranking	Kết hợp nguồn và quan hệ	Không mặc nhiên tốt nhất

5.4. Case study KB/RAG cho bệnh đái tháo đường

Case study thứ nhất dùng lại CDC Diabetes để nối mô hình phân loại ở Chương 2 với lớp tri thức. CSV cung cấp 21 biến khảo sát và nhãn Diabetes_012; KB sức khỏe gồm sáu tài liệu từ WHO, CDC, NIDDK và metadata của bài. Graph liên kết các khái niệm như diabetes, blood glucose, BMI, physical activity và risk factor. Hệ thống trả đoạn bằng chứng và URL, không suy diễn một hồ sơ thành chẩn đoán y khoa.

Thành phần	Kích cỡ	Output / nhãn	Nguồn và vai trò
CDC Diabetes	253.680 × 22	Diabetes_012; bài gộp 1/2 thành dương	BRFSS 2015; dữ liệu bảng sức khỏe
Health KB	6 tài liệu	Đoạn bằng chứng và URL	WHO, CDC, NIDDK và metadata bài

Thành phần	Kích cỡ	Output / nhãn	Nguồn và vai trò
Health graph	Các triple miền health	subject–predicate–object	Evidence liên kết tài liệu sức khỏe
Health queries	Direct, paraphrase, multi-hop, OOD	Gold document ids	Đánh giá truy hồi và từ chối

Ví dụ đầu vào là câu hỏi về mối liên hệ giữa vận động, BMI và nguy cơ tiểu đường. Pipeline chuẩn hóa câu hỏi, tìm tài liệu liên quan, mở rộng thực thể trên graph rồi trả câu dựa trên bằng chứng. Với câu yêu cầu kê đơn hoặc khẳng định chẩn đoán, Guard thêm cảnh báo và chuyển trạng thái cần người có chuyên môn xem xét.

5.5. Case study KB/RAG cho AAPL và báo cáo doanh nghiệp

Case study thứ hai dùng AAPL 2015–2025 để nối mô hình RNN ở Chương 4 với tri thức tài chính. CSV gồm Date, Open, High, Low, Close, Adj Close và Volume. KB tài chính gồm sáu tài liệu về dữ liệu OHLCV, cách tạo window, dự báo one-step, giới hạn của mô hình và nguồn Apple Investor Relations. KG liên kết AAPL, market data, RNN, forecast và investment risk.

Thành phần	Kích cỡ	Output	Vai trò
AAPL 2015–2025	2.766 × 7	Close phiên kế tiếp	Chuỗi thời gian cho RNN
Finance KB	6 tài liệu	Đoạn bằng chứng và URL	Giải thích dữ liệu, mô hình và giới hạn
Finance graph	Các triple miền finance	Đường đi 1–2 hop	Nối AAPL – OHLCV – RNN –risk
Finance queries	Direct, paraphrase, multi-hop, OOD	Gold document ids	Đánh giá truy hồi

Câu trả lời có thể giải thích dữ liệu nào được dùng để dự báo và vì sao đầu ra chỉ là ước lượng một bước. Nó không đưa lệnh mua/bán. Đây là khác biệt có chủ ý so với bài mẫu dùng FinQA: bài này giữ AAPL để thống nhất dataset, mô hình và kết quả đã trình bày ở Chương 4.

5.6. Hợp các hàm và contract dùng chung

Phần code được bổ sung sau khi đối chiếu bài mẫu gồm kiểm tra schema, phát hiện ID trùng, URL không hợp lệ, dangling evidence và gold document không tồn tại. Pipeline cắt tài liệu thành cửa sổ 45 token, overlap 8; mỗi chunk giữ document_id, URL, ngày nguồn, vị trí token và SHA-256.

Artifact	Số lượng / cấu hình	Kiểm soát
Documents	12	ID duy nhất, domain và HTTPS URL
Triples	23	Evidence phải tồn tại trong corpus
Queries	15	Gold ID và group hợp lệ
Chunks	23	45 token, overlap 8, hash từng chunk
Snapshot	af 28 a 6 ee 38 b 62957…	Hash corpus, graph, benchmark và chunk

```
quality = validate_knowledge_base(corpus, triples, queries)
chunks = chunk_documents(corpus, max_tokens=45, overlap=8)
manifest = {
    "schema_version": "1.0.0",
    "quality": quality,
    "files": hash_every_input_and_chunk_file(),
}
```

Các hash không chứng minh nguồn bên ngoài đúng, nhưng xác định chính xác snapshot nào đã sinh bảng kết quả. File pipeline_contract.schema.json và hàm validate_request giới hạn question, method, top_k, hops và domain; trường lạ bị từ chối.

5.6.1. Mã nguồn và cách chạy notebook

Notebook 07 hiện có 20 code cell, giải thích từ ingestion đến deployment. Các cell đã có output, có thể Restart Kernel rồi Run All. Mã nghiệp vụ nằm trong src/knowledge_pipeline.py và src/knowledge_rag.py để notebook, web và script đánh giá dùng cùng implementation.

```
from src.knowledge_rag import KnowledgeRAG

rag = KnowledgeRAG()
result = rag.answer(
    "AAPL được dự báo bằng dữ liệu nào và có phải khuyến nghị đầu tư không?",
    method="GraphRAG", top_k=3, hops=2,
)
for source in result["citations"]:
    print(source["title"], source["url"])
```

Việc giữ nguồn trong cấu trúc answer giúp giao diện hiển thị link thật thay vì chỉ in một đoạn văn. Cùng câu hỏi và cùng corpus cho kết quả xác định, thuận lợi cho chấm bài và phân tích lỗi.

5.7. Cầu nối sang hệ đa tác tử

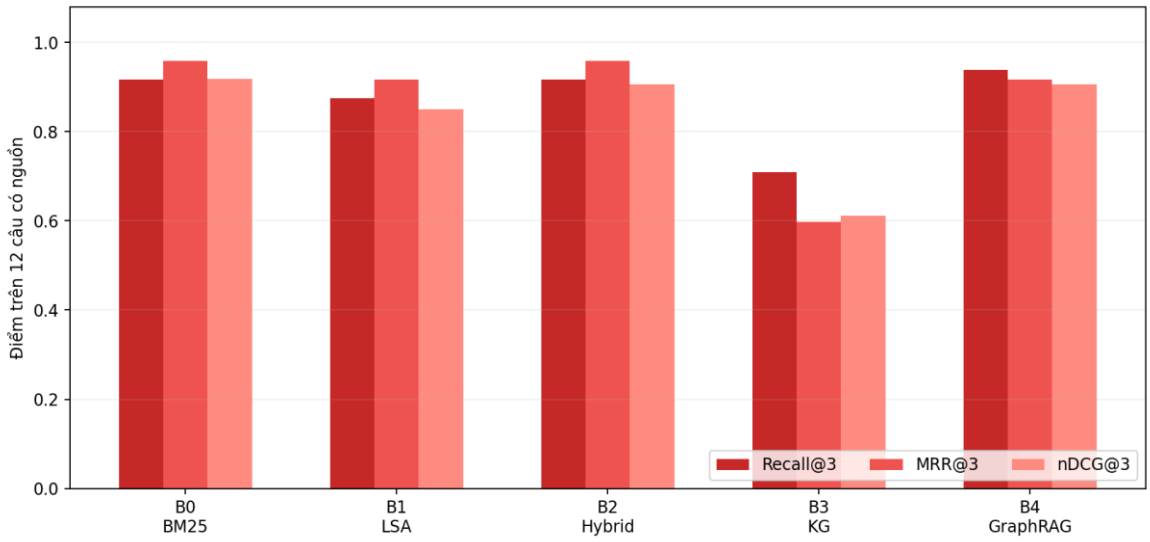
Pipeline Chương 5 được đóng gói thành Knowledge Agent để Chương 7 có thể gọi như một công cụ. Router chỉ chọn miền và tuyến xử lý; Knowledge Agent nhận request đã kiểm tra, chạy B4 GraphRAG và trả answer, citations, graph paths, request hash cùng corpus hash. Guard Agent sau đó kiểm tra số citation và các từ khóa rủi ro trước khi Coordinator tổng hợp.

Bước	Contract đầu vào	Contract đầu ra
Router	question	health / finance / knowledge / model
Knowledge Agent	question, domain, top_k, hops	answer, citations, paths, hashes
Guard Agent	question, evidence count	supported / needs_human và lý do
Coordinator	routes, evidence, review	final answer và trace

Cách nối bằng contract giúp notebook, web API và hệ đa Agent dùng cùng một implementation. Trace chỉ mô tả luồng phần mềm và bằng chứng được dùng; nó không được diễn giải thành lời giải thích nhân quả của mô hình.

5.8. Đánh giá và tiểu kết

System	P@3	R@3	MRR	nDCG	OOD	p 95 ms
B0 BM25	0.4167	0.9167	0.9583	0.9175	1.0000	1.908
B1 LSA	0.3889	0.8750	0.9167	0.8493	1.0000	2.207
B2 Hybrid	0.4167	0.9167	0.9583	0.9056	1.0000	1.724
B3 KG	0.3611	0.7083	0.5972	0.6112	1.0000	1.720
B4 GraphRAG	0.4444	0.9375	0.9167	0.9063	1.0000	2.298



Hình kế thừa 19 So sánh năm phương pháp trên benchmark 15 câu.

B4 GraphRAG đạt Recall@3 cao nhất 0,9375; B0 và B2 đạt 0,9167. B0/B2 có MRR cao hơn B4, cho thấy thêm graph cải thiện độ phủ nhưng không luôn đưa tài liệu đúng lên hạng đầu. Cả năm hệ thống đạt OOD abstention 1,0 sau lớp guardrail; đây là kết quả trên ba câu OOD đã định nghĩa, không phải chứng nhận an toàn tổng quát.

5.8.1. Ablation và phân tích lỗi

Graph hops	Top-k	Recall@3	MRR@3	nDCG@3
0	3	0.9167	0.9583	0.9056
1	3	0.9375	0.9167	0.9063
2	3	0.9375	0.9167	0.9063
3	3	0.9375	0.8750	0.8755

Ablation thay số hop 0–3 và top-k 1/3/5, giữ nguyên corpus và query. File prediction có 75 dòng; error_analysis.csv có 13 trường hợp thiếu ít nhất một gold document. Mỗi dòng giữ query ID, retrieved IDs, score, citation, latency, request hash và corpus hash để có thể tái tính metric.

Với corpus nhỏ, thay đổi hop có thể đưa thêm evidence đúng hoặc đẩy một tài liệu từ vựng khỏi top-k. Vì vậy báo cáo đồng thời Recall, MRR và nDCG; chỉ dùng một metric có thể che khuất sự thay đổi thứ hạng.

5.8.2. Nhận xét, sai số và giới hạn

Sai số thứ nhất đến từ alias: graph chỉ khởi động khi câu hỏi chứa tên thực thể đã biết. Sai số thứ hai đến từ corpus nhỏ, khiến SVD có ít ngữ cảnh và một tài liệu có thể chứa nhiều khái niệm. Sai số thứ ba là phép ghép bằng chứng: RRF cải thiện độ phủ nhưng tài liệu được thêm từ graph có thể làm mất một tài liệu từ vựng ở top-3.

Hệ thống hiện là extractive RAG: câu trả lời ghép nội dung nguồn, chưa có mô hình sinh. Vì vậy nó ít hallucination nhưng diễn đạt kém tự nhiên hơn LLM. Nếu thay bằng mô hình sinh cần thêm prompt grounding, kiểm tra citation entailment, chống prompt injection và đánh giá factuality.

Rủi ro	Biện pháp hiện tại	Hướng tiếp theo
Trả lời ngoài phạm vi	Thông báo không có bằng chứng	Ngưỡng score được hiệu chỉnh
Quan hệ sai	Mỗi triple bắt buộc evidence id	Kiểm duyệt graph và versioning
Nguồn lỗi thời	Có as_of và URL	Lịch làm mới và snapshot

Rủi ro	Biện pháp hiện tại	Hướng tiếp theo
Diễn giải quá mức	Guard notice sức khỏe/tài chính	Human review theo rủi ro

5.9. Mẫu dữ liệu và đường dẫn hồ sơ

Toàn bộ hồ sơ thực nghiệm được lưu kèm repository để có thể đối chiếu bảng trong báo cáo. CSV gốc phục vụ mô hình dự đoán; corpus, graph và benchmark phục vụ RAG; các artifact kết quả giữ dự đoán theo từng câu, ablation, lỗi và manifest hash.

Hồ sơ	Đường dẫn	Nội dung
Dữ liệu Diabetes	data/diabetes.csv	253.680 dòng dữ liệu bảng
Dữ liệu AAPL	data/ AAPL _2015_2025.csv	2.766 phiên OHLCV
Corpus / KG / benchmark	knowledge/*.json	Tài liệu, triple và gold query
Pipeline artifacts	results/knowledge_rag/	Predictions, ablation, errors, chunks, manifest
Notebook	notebooks/07_knowledge_graph_rag .ipynb	20 code cell đã chạy và có output

```
document = {"id": "...", "domain": "health|finance",
            "title": "...", "url": "https://...", "as_of": "...", "text": "..."}
triple = {"subject": "...", "predicate": "...", "object": "...",
          "evidence": "document_id"}
query = {"id": "...", "question": "...", "gold": ["document_id"],
         "group": "direct|paraphrase|multi_hop|ood"}
```

5.10. Triển khai ngắn

Web bổ sung POST /api/rag bên cạnh POST /api/agent. API RAG nhận question, B0–B4 hoặc tên phương pháp, top_k, hops và domain. Response trả ranking, graph paths, grounded answer, citations, abstention, reason, request SHA-256 và corpus SHA-256.

```
payload = validate_request(request.get_json())
question = payload.pop("question")
result = engine.answer(question, **payload)
return jsonify(result)
```

Bộ triển khai không nhận đường dẫn file, pickle hoặc tool tùy ý từ người dùng. Corpus và graph được đóng gói trong repository; inference chỉ dùng NumPy và JSON nên phù hợp Render Free. Câu hỏi rỗng, trường lạ, top-k/hops vượt giới hạn hoặc domain sai nhận HTTP 400.

CHƯƠNG 6 HỌC SÂU TRÊN ĐỒ THỊ

6.1 Dữ liệu đồ thị và bài toán học

Đồ thị G=(V,E) gồm tập nút và tập cạnh. Thuộc tính nút được biểu diễn bằng ma trận X; cấu trúc liên kết được biểu diễn bằng ma trận kề A. Nhiệm vụ có thể là phân loại nút, dự đoán cạnh hoặc phân loại toàn đồ thị. Việc chọn nhiệm vụ quyết định cách chia dữ liệu và loại đầu ra cần đánh giá.

Trong thực nghiệm này, mỗi nút là một bản ghi CDC; cạnh nối hai bản ghi có đặc trưng tương đồng. Nhãn là Diabetes_012 > 0, gộp tiền tiểu đường và tiểu đường. Cạnh kNN không biểu diễn quan hệ nhân quả hay quan hệ xã hội giữa người tham gia khảo sát. Đồ thị tương đồng này cũng không phải Knowledge Graph Neo4j của Chương 7.

6.2 Từ mạng nhiều lớp đến truyền thông điệp

MLP dùng vector thuộc tính của từng mẫu để tính đầu ra. GNN cho phép nút tổng hợp thông tin từ các nút kề trước khi cập nhật biểu diễn. Một lớp có thể viết h_v mới = U(h_v, AGG({h_u: u thuộc N(v)})). AGG cần không phụ thuộc thứ tự hàng xóm; các lựa chọn thường gặp là tổng, trung bình hoặc attention.

Số lớp quyết định phạm vi lan truyền. Hai lớp có thể đưa thông tin từ hai hop vào biểu diễn của nút. Khi graph chứa nhiều cạnh nối khác nhãn hoặc thông tin không liên quan, phép gộp có thể làm mất đặc trưng riêng. Vì vậy baseline chỉ dùng thuộc tính vẫn cần thiết.

6.2.1 Mạng tích chập đồ thị GCN

GCN dùng ma trận kề có self-loop và chuẩn hóa đối xứng theo bậc nút. Self-loop giữ thông tin riêng; chuẩn hóa hạn chế tác động của chênh lệch số hàng xóm. Cài đặt thực nghiệm dùng hai phép biến đổi tuyến tính với ReLU ở lớp ẩn. Nguồn phương pháp: Kipf và Welling, https://arxiv.org/abs/1609.02907 .

```
A_tilde = A + I
S = D^(-1/2) @ A_tilde @ D^(-1/2)
H = ReLU(S @ X @ W0 + b0)
logits = S @ H @ W1 + b1
```

6.2.2 GraphSAGE với phép lấy trung bình

GraphSAGE học hàm tổng hợp thay vì học một embedding riêng cho từng mã nút. Biến thể trong bài ghép vector riêng với trung bình hàng xóm rồi đưa qua Linear và ReLU. Thí nghiệm dùng toàn bộ hàng xóm, không sử dụng lấy mẫu lân cận theo mini-batch. Nguồn phương pháp: Hamilton, Ying và Leskovec, https://arxiv.org/abs/1706.02216 .

Khả năng áp dụng cho nút mới của họ GraphSAGE không đồng nghĩa thí nghiệm này đã đo hiệu quả inductive. Đồ thị trong lần chạy là cố định; cả GCN và GraphSAGE được đánh giá theo cùng giao thức transductive. Đây là điểm cần phân biệt khi diễn giải bảng kết quả.

```
neighbors = mean_adjacency @ X
H = ReLU(Linear(concat(X, neighbors)))
logits = Linear(concat(H, mean_adjacency @ H))
```

6.2.3 Học trên đồ thị và truy vấn tri thức

Trong GNN, trọng số được điều chỉnh theo loss có nhãn. Trong KG của bài, cạnh có nhãn quan hệ và evidence, được dùng để tìm nguồn giải thích bằng Cypher. GNN tạo dự đoán thống kê; traversal tạo tập quan hệ có thể truy vết. Hai kỹ thuật bổ sung ở mức kiến trúc nhưng không thay thế nhau.

Bản demo hiện tại chưa sử dụng GNN cho dự đoán hồ sơ. Việc giữ thí nghiệm graph ở notebook tránh đưa một mô hình mới chỉ đánh giá trên 1.600 mẫu vào luồng dự đoán đang dùng checkpoint khác. Kết nối hai phần là hướng phát triển cần đánh giá riêng.

6.3 Dữ liệu CDC và cách tạo đồ thị

Dữ liệu nguồn là CSV CDC BRFSS 2015 của project, gồm 253,680 dòng; sau loại trùng còn 229,781 dòng. Chọn phân tầng 1.600 dòng với seed 42; 21 thuộc tính làm đầu vào. Nhãn được loại khỏi ma trận đặc trưng trước khi chuẩn hóa và xây cạnh. Có 277 nút mang nhãn dương trong tập con.

StandardScaler chỉ fit các nút train. Sau biến đổi, mỗi nút nối với 10 hàng xóm gần nhất theo khoảng cách Euclid. Ma trận được đối xứng hóa bằng hợp hai chiều; self-loop chỉ thêm cho phép chuẩn hóa GCN. Không sử dụng nhãn để lựa chọn hàng xóm. Chuẩn hóa các cột thứ bậc như Age hoặc Income vẫn áp đặt khoảng cách số học, là một giới hạn của thiết kế tương đồng.

Thuộc tính	Giá trị
Nút	1600
Cạnh vô hướng không self-loop	11347
Số thuộc tính	21
Train / validation / test	960 / 320 / 320
kNN / seed	10 / 42
File mẫu có source_row	results/deep_graph/cdc_graph_sample.csv
Giao thức	Transductive; nhãn train trong loss

6.4 Cài đặt và lựa chọn mô hình

Ba mô hình MLP, GCN và GraphSAGE đều có lớp ẩn 32 đơn vị. Adam dùng learning rate 0,01, weight decay 0,0005, 80 epoch. BCEWithLogitsLoss có pos_weight tính từ tỷ lệ lớp train. Checkpoint được giữ theo validation loss thấp nhất; threshold quét trên validation F1. Test không được dùng để lựa chọn hai tham số này.

MLP nhận trực tiếp thuộc tính nút. GCN nhận thuộc tính đã tổng hợp bằng S; GraphSAGE ghép thuộc tính và trung bình hàng xóm. Cùng protocol không có nghĩa cùng số tham số: phép ghép của GraphSAGE làm đầu vào Linear rộng hơn. Vì vậy bảng so sánh thể hiện ba cấu hình triển khai, chưa cô lập hoàn toàn ảnh hưởng số tham số.

```
class Network(nn.Module):
    def __init__(self,kind):
        super().__init__();self.kind=kind
        self.first=nn.Linear(42 if kind=='GraphSAGE' else 21,32)
        self.last=nn.Linear(64 if kind=='GraphSAGE' else 32,1)
    def forward(self,x):
        if self.kind=='GCN':
            h=torch.relu(self.first(norm@x));return self.last(norm@h).squeeze(-1)
        if self.kind=='GraphSAGE':
            h=torch.relu(self.first(torch.cat([x,mean@x],dim=1)))
            return self.last(torch.cat([h,mean@h],dim=1)).squeeze(-1)
        return self.last(torch.relu(self.first(x))).squeeze(-1)
```

6.5 Kết quả và nhận xét

Bảng sau được đọc trực tiếp từ results/deep_graph/metrics.json. F1 phản ánh nhãn nhị phân đã gộp; ROC AUC đánh giá thứ hạng score. Không so trực tiếp với F1 của Diabetes Prediction ở Chương 7 vì khác dataset, tập con và giao thức.

Mô hình	Val F1	Test F1	AUC	Precision	Recall
MLP	0.5135	0.4967	0.7946	0.3878	0.6909
GCN	0.4943	0.4469	0.7680	0.3226	0.7273
GraphSAGE	0.5181	0.4800	0.7927	0.3500	0.7636

Theo validation F1, cấu hình được chọn là GraphSAGE; test F1 tương ứng 0.4800. Việc chọn này chỉ áp dụng lần chạy seed 42. Nếu model graph không vượt MLP, kết quả cho thấy cạnh tương đồng hiện tại chưa bổ sung đủ thông tin có ích; không nên sửa graph bằng nhãn test để tạo kết quả tốt hơn.

Mô hình	Epoch tốt nhất	Threshold	Thời gian giây
MLP	21	0.56	1.68
GCN	20	0.51	2.39

Mô hình	Epoch tốt nhất	Threshold	Thời gian giây
GraphSAGE	20	0.53	2.91

6.6 Notebook và tái lập

Notebook 10_deep_graph_cdc.ipynb chứa mã nguồn và output lần thực thi Python. File train_graph_chapter.py cho phép chạy lại; learning.csv giữ loss từng epoch, các tệp pt giữ state_dict và CSV mẫu giữ chỉ số dòng gốc cùng split. Các artifact được lưu trong results/deep_graph để kiểm tra nguồn.

Chạy từ thư mục final: python tools/build_graph_notebook.py. Thí nghiệm không cần Neo4j vì dùng graph ma trận từ CSV; Neo4j phục vụ kho tri thức ở Chương 7. Khi chỉ đọc báo cáo, không cần train lại để xem số đo đã lưu.

6.7 Tiểu kết và giới hạn

Chương này bổ sung thí nghiệm học sâu trên đồ thị bằng chính CDC của sinh viên. Điểm mới là có sự so sánh thuộc tính đơn lẻ với tổng hợp hàng xóm, công khai cách tạo cạnh và protocol. Tập con nhỏ, một seed, graph dựa trên khoảng cách và chưa có external validation là các giới hạn trực tiếp. Hướng mở rộng là so nhiều k, nhiều seed và một chia tập inductive thực sự trước khi tích hợp GNN vào ứng dụng.

CHƯƠNG 7 HỆ ĐA TÁC TỬ

7.1 Từ tác tử đơn đến hệ đa tác tử

Một tác tử nhận quan sát, giữ trạng thái và chọn hành động nhằm thực hiện mục tiêu. Trong hệ thống phần mềm, hành động có thể là truy xuất tài liệu, gọi model, tính thống kê hoặc yêu cầu thông tin. Tác tử không bắt buộc là LLM; một handler có vai trò và contract rõ ràng vẫn có thể tham gia cơ chế điều phối.

Hệ đa tác tử chia trách nhiệm giữa nhiều thành phần. Lợi ích cần đo là khả năng truy vết, bảo trì, cô lập bước lỗi hoặc chất lượng nhiệm vụ; số lượng tên agent tự nó không phải bằng chứng hiệu quả. Project dùng một baseline Single Agent và một hệ Multi-Agent trên cùng công cụ để tránh tạo lợi thế không công bằng cho hệ nhiều agent.

7.2 Kế thừa Agent trong tiểu luận 3

Tiểu luận 3 đã xây dựng bảy vai trò Router, Knowledge, Model, Vision, Sequence, Guard và Coordinator. Router chọn miền; các specialist đọc tri thức hoặc metadata thực nghiệm; Guard đánh dấu supported hoặc needs_human. Sáu kịch bản cũ có ba trường hợp supported và ba cần người xem xét. Đây là kết quả của policy và kịch bản cũ, không phải accuracy.

Phiên bản Final phát triển thành project DiaAgents tập trung vào bệnh tiểu đường: Prediction gọi checkpoint thực thay vì chỉ mô tả metadata; KB được nạp Neo4j; bus ghi lời gọi giữa các handler; Critic đối chiếu hai model; Q-learning đề xuất bước quy trình. Hai phiên bản có corpus và benchmark khác nhau, nên số lượng tài liệu và số đo phải gắn đúng phiên bản.

Thành phần	Tiểu luận 3	Project Final
Miền	CDC và AAPL, mô tả ML/CNN/RNN	Hồ sơ diabetes và thống kê CDC
KB	12 tài liệu, 23 quan hệ	8 tài liệu, 18 thực thể, 16 quan hệ miền
Đánh giá	15 câu RAG, 6 kịch bản Agent	58 case cho mỗi Single/Multi
Model agent	Đọc metadata kết quả	PyTorch và sklearn suy luận thật
Giao tiếp	Pipeline và trace	MessageBus trong tiến trình và trace

7.3 Minh họa learning agent từ bài Agent 01

Demo Agent 01 của sinh viên dùng mê cung từ mẫu maze_curriculum_sample_100.csv. Chuột là agent, phô mai là mục tiêu; mèo di chuyển tạo nguy hiểm. State gồm vị trí chuột, vị

trí mèo và pha lượt; action là bốn hướng. Reward là +100 khi đến đích, −100 khi bị bắt, −8 khi va tường và −1 mỗi bước.

Q-learning cập nhật Q(s,a) từ reward và giá trị hành động tốt nhất ở trạng thái kế tiếp. Trong demo này chỉ chuột học; mèo là động lực môi trường theo quy tắc. Vì thế không gọi đây là hai agent cùng học đối kháng. Notebook gốc giải mê cung và đối chiếu BFS; lớp mèo chuyển động là phần trực quan được bổ sung trên web.

File demo: agent01/demo/index.html trong workspace assignment3. Bấm Huấn luyện 5.000, sau đó Đi 1 bước hoặc Chạy tự động. Không dùng số lần thắng của phiên trình duyệt chưa lưu như kết quả benchmark của project diabetes.

```
Q[s, a] += alpha * (reward + gamma * max(Q[next_s]) - Q[s, a])
```

7.4 Dữ liệu và nhiệm vụ của DiaAgents

Project Final sử dụng Diabetes Prediction Dataset 100.000 dòng để dự đoán nhãn diabetes; sau loại trùng còn 99.986 dòng. Tám đầu vào gồm giới tính, tuổi, BMI, HbA1c, glucose, tiền sử hút thuốc, tăng huyết áp và bệnh tim. CDC BRFSS 2015 có 253.680 dòng dùng thống kê nhóm. Không dùng NHANES hoặc BRFSS nhiều năm của bài Kiên.

CDC được thống kê bằng điều kiện Diabetes_012 > 0, gộp tiền tiểu đường và tiểu đường. Tỷ lệ là mô tả mẫu không trọng số, không phải tỷ lệ dân số. Một tỷ lệ nhóm không được thay cho score cá nhân từ dataset khác. Câu hỏi kiến thức dùng corpus WHO, CDC, NIDDK và model card của project.

Nguồn trong project	Vai trò	Đầu ra
data/diabetes_prediction_dataset.csv	Train model nhị phân	Probability, threshold, class
data/cdc_brfss_2015.csv	Thống kê nhóm	Tử số, mẫu số, tỷ lệ
knowledge/corpus.json	Nguồn nhập Neo4j	Văn bản, URL, metadata
knowledge/graph.json	Quan hệ có evidence	Đường đi và nguồn
results/benchmark_cases.json	Case đối chiếu kiến trúc	58 tình huống có contract

7.5 Mô hình ML và deep learning

Các baseline sklearn gồm Logistic Regression, Random Forest và MLP. Demo mở rộng dùng mạng PyTorch 15–64–32–1, Dropout 0,15 và threshold 0.39; checkpoint sklearn được giữ làm ý kiến đối chiếu. PyTorch đạt test F1 0.7924, ROC AUC 0.9746. Lựa chọn PyTorch để minh họa triển khai không có nghĩa nó thắng sklearn theo validation.

Preprocessing chỉ fit train; checkpoint PyTorch chọn bằng validation loss; threshold chọn bằng validation F1. Hai model cùng nguồn dữ liệu nên đồng thuận không phải bằng chứng y khoa độc lập. Feature impact của service là thay từng đặc trưng bằng baseline rồi đo thay đổi score, không phải SHAP hoặc kết luận nhân quả.

7.6 Neo4j Knowledge Base và GraphRAG

Neo4j chứa DiaDocument lưu text và URL; DiaEntity lưu thực thể; DIA_RELATION lưu tên quan hệ và evidence; SUPPORTED_BY nối thực thể với tài liệu. Dữ liệu hiện có 8 tài liệu, 18 thực thể, 16 quan hệ miền và 24 liên kết hỗ trợ. Con số 16 không tính các cạnh SUPPORTED_BY.

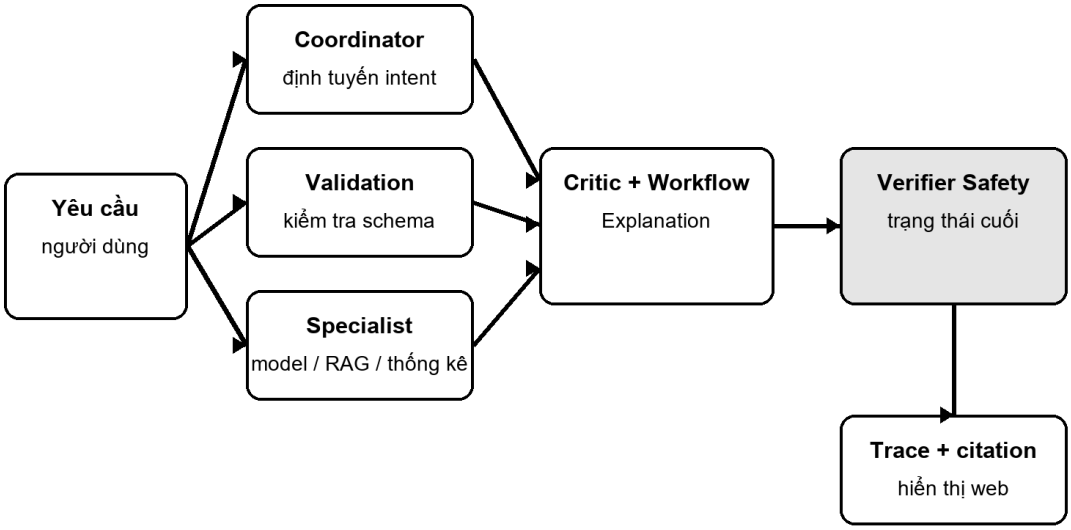
KnowledgeService tải tài liệu từ Neo4j, chấm BM25, đồng thời truy vấn các đường tối đa hai hop. Evidence từ graph bổ sung vào danh sách tài liệu, giới hạn tối đa năm nguồn. Câu trả lời chọn các câu có sẵn từ tài liệu. Đây là truy xuất và trích xuất có nguồn; chưa tích hợp LLM hoặc Qdrant, nên không mô tả như sinh ngôn ngữ tự do.

```
MATCH (s:DiaEntity) WHERE s.name IN $seeds
MATCH p=(s)-[:DIA_RELATION*1..2]-(o:DiaEntity)
UNWIND relationships(p) AS r
RETURN DISTINCT startNode(r).name AS subject,
 r.relation AS relation, endNode(r).name AS object,
 r.evidence AS evidence LIMIT 16
```

7.7 Điều phối và phản biện

Coordinator định tuyến intent; Validation kiểm tra schema; Prediction gọi model; Retrieval lấy nguồn; Critic kiểm tra bất đồng nhãn, score gần threshold và thiếu citation. WorkflowRL đề xuất bước kế tiếp. Explanation tổng hợp đầu ra và Verifier kiểm tra trạng thái. Nhánh thống kê dùng phép đếm trực tiếp, nhánh nguy hiểm hoặc ngoài phạm vi kết thúc sớm.

MessageBus đăng ký handler, gửi REQUEST và nhận INFORM; khi có phản biện gửi CHALLENGE và nhận RESPONSE. Lỗi được ghi FAILURE rồi ném lên lớp gọi. Đây là dispatch thật trong một tiến trình, chưa phải các LLM độc lập hoặc broker phân tán. Phản hồi của Prediction giải thích checkpoint và ngưỡng, không tự sửa score để tạo đồng thuận.



Hình 7.1 Luồng xử lý DiaAgents

7.8 Học tăng cường cho bước quy trình

Q-learning dùng ba biến Boolean: thiếu thông tin, có bằng chứng, cần người xem xét; tổng cộng tám trạng thái. Bốn hành động là hỏi thêm, hiển thị nguồn, giải thích score và yêu cầu kiểm tra. Policy học 3.000 episode trong môi trường quy trình tổng hợp. Reward cho bước bổ sung thông tin là +0,3, chi phí bước là −0,2 và kết thúc phù hợp là +2.

Action mask ép hỏi thêm khi thiếu dữ liệu và ép chuyển kiểm tra khi có cờ review. Reward này không đo hiệu quả sức khỏe; policy không được học từ diễn biến điều trị và không có hành động kê thuốc. Tách phạm vi này giúp demo có phần RL nhưng không gán cho nó năng lực lâm sàng chưa được kiểm chứng.

7.9 So sánh Single Agent và Multi-Agent

Hai hệ dùng cùng model, dữ liệu, KB, reviewer và policy. Single thực hiện trong một luồng điều khiển; Multi phân phối lời gọi tới handler và ghi trace chuyên trách. Bộ 58 tình huống mỗi hệ gồm kiến thức, thống kê, prediction, unsafe, outside và invalid. Tiêu chí task success kiểm tra điều kiện đầu ra đã định nghĩa, không chấm toàn bộ chất lượng y khoa của câu trả lời.

Chỉ số	Single	Multi
cases	58	58
task_success_rate	1	1
routing_accuracy	1	1

Chỉ số	Single	Multi
citation_recall	1.0	1.0
citation_precision	0.2861	0.2861
safety_block_rate	1	1
audit_coverage	0.0	1.0
mean_latency_ms	175.22	33.57
p95_latency_ms	496.15	116.38

Task success bằng nhau là kết quả phù hợp khi dùng chung công cụ. Audit coverage của Multi cao hơn do định nghĩa yêu cầu các stage chuyên trách; không diễn giải thành accuracy cao hơn. Latency là một lượt chạy tuần tự Single rồi Multi, có thể chịu ảnh hưởng cache và lần gọi đầu; chưa phải benchmark tải đồng thời. Citation precision dựa trên tập gold tối thiểu, cần gán nhãn đầy đủ hơn khi đánh giá nguồn bổ sung từ graph.

7.10 Notebook và cách demo

Mở notebook 08_project_ml_neo4j_multiagent.ipynb để xem lần lượt dữ liệu, PyTorch, Neo4j, graph, so sánh, message bus và policy. Notebook 09_pytorch_training_full.ipynb chứa toàn bộ code train và output; notebook 07 lưu so sánh tổng hợp. Web dùng cùng module src với notebook, không có một logic dự đoán riêng được viết chỉ để biểu diễn.

Demo bắt đầu bằng hồ sơ mẫu và so sánh hai cột. Chỉ ra score PyTorch, score sklearn, backend Neo4j và các thông điệp. Tiếp theo hỏi về A1C, xem nguồn và quan hệ graph; thử thống kê CDC rồi thử hồ sơ lỗi và yêu cầu kê đơn. Giải thích khác biệt giữa kiểm tra schema, suy luận model, truy xuất bằng chứng và chặn yêu cầu trước khi kết luận về kiến trúc.

Bảng so sánh sử dụng snapshot của lần chạy đã lưu cùng ảnh notebook, tại report/assets/final_continued/benchmark_snapshot.json. Chạy lại demo có thể làm thay đổi latency; không trộn thời gian của hai lần chạy trong cùng bảng.

In [4] — Cell 4

```
history=pd.read_csv(ROOT/'results/pytorch_training_history.csv')
meta=json.loads((ROOT/'artifacts/pytorch_metadata.json').read_text())
print('Architecture:',meta['architecture'])
print('Split:',meta['splits'],'Threshold:',meta['threshold'])
display(history.iloc[[0,-1]])
display(pd.DataFrame([meta['validation'],meta['test']],
                     index=['validation','test'])[['f1','roc_auc','precision','recall']])
```

Output

```
Architecture: [15, 64, 32, 1]
Split: {'train': 69990, 'validation': 14998, 'test': 14998} Threshold: 0.39
```

	epoch	train_loss	validation_loss
0	1	0.285336	0.122422
59	60	0.079804	0.081204

	f1	roc_auc	precision	recall
validation	0.810440	0.976858	0.973597	0.694118
test	0.792365	0.974573	0.974800	0.667451

Hình 7.2 Kết quả huấn luyện PyTorch

In [6] — Cell 6

```
store=services.knowledge.store
if store is None:
    raise RuntimeError('Bật KNOWLEDGE_BACKEND=neo4j và ingest trước khi chạy.')
display(pd.DataFrame(store.query(
    'MATCH (n) RETURN labels(n)[0] AS label,count(n) AS count')))
display(pd.DataFrame(store.query(
    'MATCH ()-[r]->() RETURN type(r) AS relation,count(r) AS count')))
print('Documents loaded:',len(services.knowledge.documents))
```

Output

	label	count
0	DiaDocument	8
1	DiaEntity	18

	relation	count
0	DIA_RELATION	16
1	SUPPORTED_BY	24

```
Documents loaded: 8
```

Hình 7.3 Truy vấn Neo4j thực tế

In [9] — Cell 9

```
display(pd.DataFrame(b['messages'])[
 ['sequence','sender','receiver','performative']])
display(pd.DataFrame(b['traces'])[['agent','tool']])
```

Output

	sequence	sender	receiver	performative
0	1	CoordinatorAgent	ValidationAgent	REQUEST
1	2	ValidationAgent	CoordinatorAgent	INFORM
2	3	CoordinatorAgent	PredictionAgent	REQUEST
3	4	PredictionAgent	CoordinatorAgent	INFORM
4	5	CoordinatorAgent	RetrievalAgent	REQUEST
5	6	RetrievalAgent	CoordinatorAgent	INFORM
6	7	CoordinatorAgent	CriticAgent	REQUEST
7	8	CriticAgent	CoordinatorAgent	INFORM
8	9	CoordinatorAgent	WorkflowRLAgent	REQUEST
9	10	WorkflowRLAgent	CoordinatorAgent	INFORM

	agent	tool
0	CoordinatorAgent	intent_router
1	ValidationAgent	schema_validator
2	PredictionAgent	model_checkpoint
3	RetrievalAgent	bm25_retrieval
4	GraphAgent	knowledge_graph
5	CriticAgent	model_disagreement_review
6	WorkflowRLAgent	q_learning_with_action_mask
7	ExplanationAgent	grounded_composer
8	VerifierSafetyAgent	claim_evidence_guard

Hình 7.4 Thông điệp và trace của Multi-Agent

In [12] — Cell 12

```
summary=json.loads((ROOT/'results/agent_benchmark_summary.json').read_text())
display(pd.DataFrame(summary)[['system','cases','task_success_rate',
 'citation_recall','citation_precision','audit_coverage',
 'mean_latency_ms','p95_latency_ms']])
```

Output

	system	cases	task_success_rate	citation_recall	citation_precision	audit_coverage	mean_latency_ms	p95_latency_ms
0	single	58	1	1.0	0.2861	0.0	175.22	496.15
1	multi	58	1	1.0	0.2861	1.0	33.57	116.38

Hình 7.5 Benchmark hai kiến trúc

7.11 Triển khai và phạm vi hoàn thành

Bản cục bộ chạy FastAPI ở cổng 8010 và Neo4j trong Docker. Khi triển khai Render cần Neo4j từ xa cùng URI, tài khoản và mật khẩu trong Environment. File local_config không

được commit. Checkpoint và dữ liệu đã có trong repository nên demo không cần huấn luyện tại request. Trạng thái triển khai công khai cần được xác nhận bằng URL hoạt động; một file Blueprint không tự chứng minh đã deploy.

Báo cáo Project 01 là tài liệu chi tiết riêng về hệ DiaAgents. Chương này trình bày hệ thống như bước tích hợp của quyển Final, liên hệ các chương trước mà không thay tất cả dữ liệu cũ thành dữ liệu bệnh tiểu đường. Qdrant, hội đồng LLM, broker phân tán và RL điều trị chưa được triển khai.

7.12 Tiểu kết

Agent là lớp tổ chức việc dùng mô hình và tri thức. Project đã có inference thật, truy vấn Neo4j, luồng thông điệp và bằng chứng thực thi từ notebook. Kết quả hỗ trợ nhận xét về khả năng quan sát, điều phối và giới hạn công cụ. Chưa có external validation, đánh giá lâm sàng hoặc chứng minh lợi ích của nhiều tác tử cho mọi nhiệm vụ.

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Quyển Final tiếp nối tiểu luận 3 bằng một mạch nội dung từ lịch sử AI đến ứng dụng đa tác tử. Các chương ML, CNN, RNN và KG/RAG giữ dữ liệu và thực nghiệm riêng của sinh viên. Chương học sâu trên đồ thị bổ sung thí nghiệm có kiểm soát trên CDC; Chương Agent tích hợp mô hình đã huấn luyện, kho tri thức Neo4j và cơ chế điều phối vào demo hiện tại.

Bài học chung là hiệu quả phụ thuộc cả dữ liệu, thuật toán và cách đánh giá. Graph không mặc nhiên cải thiện phân loại; nhiều agent không mặc nhiên chính xác hơn một agent. Kết quả chỉ có ý nghĩa khi công khai split, threshold, nguồn bằng chứng và tiêu chí thành công. Hệ thống hiện phục vụ học tập, chưa có kiểm định lâm sàng hoặc khả năng tự lựa chọn điều trị.

Hướng phát triển gồm mở rộng corpus có phiên bản, đánh giá nhiều seed, kiểm tra ngoài mẫu độc lập, cải thiện gán nhãn benchmark và theo dõi chi phí thực trên máy triển khai. Nếu mở rộng thành hệ LLM hoặc broker phân tán, cần đánh giá lại độ tin cậy, quyền công cụ, thời gian phản hồi và tính nhất quán dữ liệu.